系统分析与设计

SYSTEM ANALYSIS AND DESIGN

2026 年 10 月

ADC 牙科连锁信息管理系统

ADC

ANDREW'S DENTAL CHAIN

安德鲁牙科连锁

面向规模化扩张的数据设计

报告信息

姓名	学号
马博晨	8210241903
班级	指导教师
信管 T2401 班	毕文杰

报告结构

A 业务建模	B 需求	C 分析	D 设计

从业务建模到数据设计的一次完整实践：愿景、业务图、需求规约、分析图与数据库图表，以及数据层、业务层与表示层的分层设计交付。

目　录

1. 拟开发项目概述 1

2. 愿景 1

2.1 业务现状与主要问题 1

2.2 愿景表述 2

2.3 度量指标的选取说明 2

3. 业务建模 3

3.1 业务执行者与业务工人 3

3.2 业务用例图 3

3.3 现状业务序列图 4

3.4 改进后业务序列图 5

3.5 改进前后对照 7

4. 需求 7

4.1 需求获取：五种方法与本案的取舍 7

4.2 需求是涉众利益的平衡 8

4.3 系统执行者 9

4.4 系统用例图 9

4.5 系统用例清单 10

4.6 用例规约 10

5. 分析：从类图到序列图与状态机图 15

5.1 分析类图的三种构造型 16

5.2 实体类的命名自检 16

5.3 实体类清单与概念属性 17

5.4 类之间的关系与多重性 18

5.5 分析类图（一）：患者与诊疗域 20

5.6 分析类图（二）：组织与人员域 21

5.7 分析类图（三）：边界类与控制类 22

5.8 分析序列图（一）：UC-02 受理预约 23

5.9 分析序列图（二）：UC-03 记录就诊与治疗 25

5.10 分析状态机图：预约与结算单 26

6. 设计：数据层、业务层与表示层 27

6.1 类图到表的映射规则 27

6.2 泛化怎么落：三种做法与本案的选择 28

6.3 自反关系与值表 28

6.4 数据访问方式：四种模式与本案的选择 29

6.5 每条关联的外键落点 30

6.6 数据库表清单：16 张表的主键与外键 31

6.7 数据库图表（一）：组织与人员域 32

6.8 数据库图表（二）：患者与诊疗域 33

6.9 数据字典 34

6.10 建表脚本 38

6.11 范式检查 41

6.12 设计约束与索引安排 42

6.13 精化业务层：从责任到操作 43

6.14 精化表示层：边界类的界面责任 45

1. 拟开发项目概述

本项目以安德鲁牙科连锁（Andrew's Dental Chain，以下简称 ADC）为拟开发项目的目标组织。ADC 在中国广东省经营 15 家牙科诊所，提供洗牙、牙齿修复、X 光、抛光、植牙、牙桥、牙冠等诊疗服务。目前各诊所基本上以“独立诊所的集合”模式运行，依赖总部人工监督管理，患者档案、预约登记、治疗记录与结算单分散在各诊所以纸质表单和本地表格的形式保存。

ADC 计划在未来三年内把诊所数量翻一番（15 家增至 30 家）。随着规模扩大，这种“各自为政 + 人工监督”的架构暴露出三个悬而未决的问题：各地患者的治疗标准是否统一？各诊所的患者取消率是否存在差异？ADC 聘用的牙医是否具备符合业务需求的技能组合？与此同时，ADC 提出了一项按净利润的 10% 分配激励基金的制度，却缺乏按季度跟踪每位牙医收入、成本与运营费用的数据基础，制度难以落地。

首席执行官兼首席牙医 Andrew Chen 持有公司 60% 的股权，拥有绝对决策权；但他清醒地意识到：要让连锁在快速扩张中不失控，必须建立基于标准化、高质量数据库的信息系统，这是防止业务失控的必要条件。

因此，本项目拟开发的目标系统是 **ADC 牙科连锁信息管理系统**：一个以“患者—预约—就诊—治疗—结算单”为主线、覆盖牙医资质与诊所资源的连锁级关系型信息系统。本报告按课程建模工作流 **A 业务建模** → **B 需求** → **C 分析** → **D 设计**的顺序推进，依次交付愿景、业务建模工件、需求工件、分析工件（分析类图、分析序列图、分析状态机图）与设计工件（数据层、业务层、表示层）。

2. 愿景

2.1 业务现状与主要问题

把 ADC 当前的诊疗业务“拍成片子”，可以看到三类反复出现的问题：

- **信息散落在各诊所，总部只能事后人工汇总。**患者档案、预约登记、治疗记录、结算单分别保存在 15 家诊所的纸质表单与各自的本地表格里；总部要了解经营情况，只能让各诊所人工上报、再手工合并，既慢又失真。
- **管理口径不统一，风险看不见。**取消就诊是牙科行业的重要危险信号，需要迅速采取应对措施；但各诊所以不同方式记录“是否到诊”，总部既无法比较各诊所的取消率，也无从判断各地的治疗标准是否统一。
- **牙医的价值与能力说不清。**牙医并不专属任何一家诊所，而是按所需技能在多家诊所间轮转；但“谁具备哪类资质、谁创造了多少净利润”没有系统记录，激励制度与人才招募都缺乏依据。

2.2 愿景表述

愿景要同时满足三个条件：**来自组织最重要的涉众、表述简洁、可以量化度量**。本案最重要的涉众是持有 60% 股权、拥有绝对决策权的 Andrew Chen，因此愿景以他的诉求为准，而不是以 IT 部门或某一家诊所的诉求为准；表述压缩成一张表、一段话；度量落到“取消率”这个能按季度算出具体数值的指标上，选取理由见下节。

目标系统愿景

系统	ADC 牙科连锁信息管理系统
目标组织	安德鲁牙科连锁（ADC）
老大	首席执行官兼首席牙医 Andrew Chen
目标	在诊所数量三年翻番的同时，使总部对各家诊所的患者取消率、治疗标准执行与牙医产能保持一致、可比较的可视与可控
度量指标	全连锁季度平均患者取消率 $=\frac{\sum \text{未到诊预约数}}{\sum \text{预约数}}$（单位：%），并按诊所、按牙医拆分
对比时间	扩张启动前的一个完整财年（15 家诊所基线）
改进值	全连锁平均取消率下降 25% 以上；新增诊所开业首季度即达到与成熟诊所一致的取消率水平

2.3 度量指标的选取说明

愿景目标是**组织指标**，不是系统功能，也不是系统的性能指标。选择“患者取消率”作为头号指标，理由有三：

1. 它直接对应老大最关心的事——案例明确指出，高取消率是危险信号，必须迅速采取应对措施；取消率同时反映服务质量与产能浪费，是连锁扩张中最需要尽早发现的管理缺口。
2. 它可度量：每次预约是否到诊都能以布尔字段记录（初始为 0/false，就诊后更新为 1/true），按季度汇总即可得到基线与改进值，并可进一步按诊所、按牙医下钻。
3. 它能牵引需求排序：凡是支撑“记录就诊状态、按诊所与牙医统计、共享患者档案、按技能匹配牙医”的用例优先做，与标准化管理无关的花哨功能往后放。

在权衡上，本系统暂时优先“看得清”（取消率、治疗标准执行、牙医业绩可统计），把“看得细”（如单次治疗的临床影像管理）放在次要位置；需要说明的是，若其他指标发生变化（例如扩张后牙医产能成为瓶颈），应在后续迭代中重新排序。

3. 业务建模

业务建模要回答两个问题：**对外**，组织为外部提供了什么价值（业务用例图）；**对内**，组织内部各系统如何协作实现这些价值（业务序列图）。前者是“价值的集合”，后者是“系统的集合”。

3.1 业务执行者与业务工人

业务执行者是在组织之外、与组织交互的人群或机构（注意：是责任边界，不是物理边界）：

- **患者**：诊疗服务的接受方，是 ADC 存在价值的来源，也是取消率指标的主体。
- **医疗耗材供应商**：向诊所供应一次性器械、麻醉剂、修复材料等耗材。
- **支付 / 保险机构**：第三方结算机构，ADC 通过其完成诊疗费用的收付与理赔。

业务工人是组织内部承担责任的“人肉系统”，包括**牙医**、**前台接待员**、**诊所主管**与**总部管理者**。纸质患者登记表、各诊所的本地表格台账、纸质病历与手写结算单是组织内部的**业务实体**（非人系统），它们在业务序列图中与人和机构同等对待——这正是本次改进要替换的对象。

需要说明：ADC 的五位合伙人（除 Chen 外）是公司的所有者，与公司之间是股权关系，不参与诊疗流程本身，故不作为业务执行者纳入本次业务用例图。

3.2 业务用例图

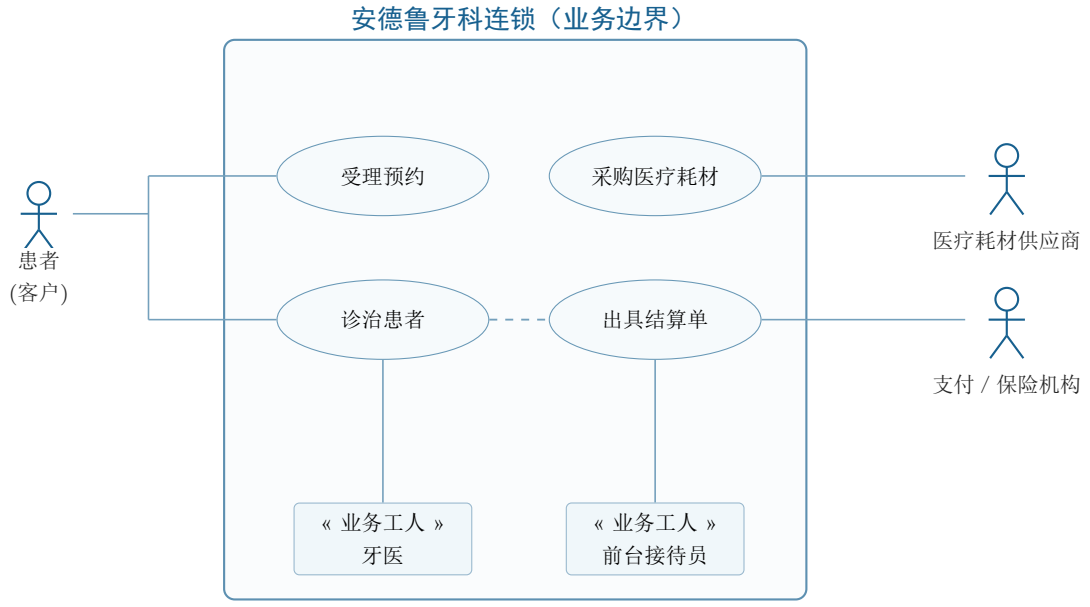


图 1　业务用例图：安德鲁牙科连锁对外提供的价值

图 1 说明 图中矩形是 ADC 的业务边界，椭圆是从组织外部视角看到的业务用例。**诊治患者**是核心业务用例：患者把“把牙治好”这件事交给 ADC 完成，ADC 由此获得诊疗收入，这是时光倒流三百年也依然成立的价值。**受理预约**是诊疗的入口——患者必须先把“某个时间请某位牙

医看诊”这件事交给组织，诊治才可能发生。**出具结算单**是价值兑现的环节，患者与支付 / 保险机构通过它完成费用结算，这也是 ADC 希望推进标准化的地方（图中以虚线表示它依赖诊治产生的治疗数据）。**采购医疗耗材**位于图的右侧，服务的对象是供应商一侧的订单与货款关系。连线已按执行者各自归口，表示这些价值由牙医与前台接待员分别承担并共同实现——注意业务工人画在业务边界**内部**，他们是组织的人肉零件，而患者、供应商、支付 / 保险机构在边界之外。

3.3 现状业务序列图

选取与愿景最相关的流程片段：“患者来电预约 → 前台登记 → 患者到诊 → 牙医治疗 → 手写记录 → 手写结算 → 收款 → 月末人工汇总”的一次完整就诊。

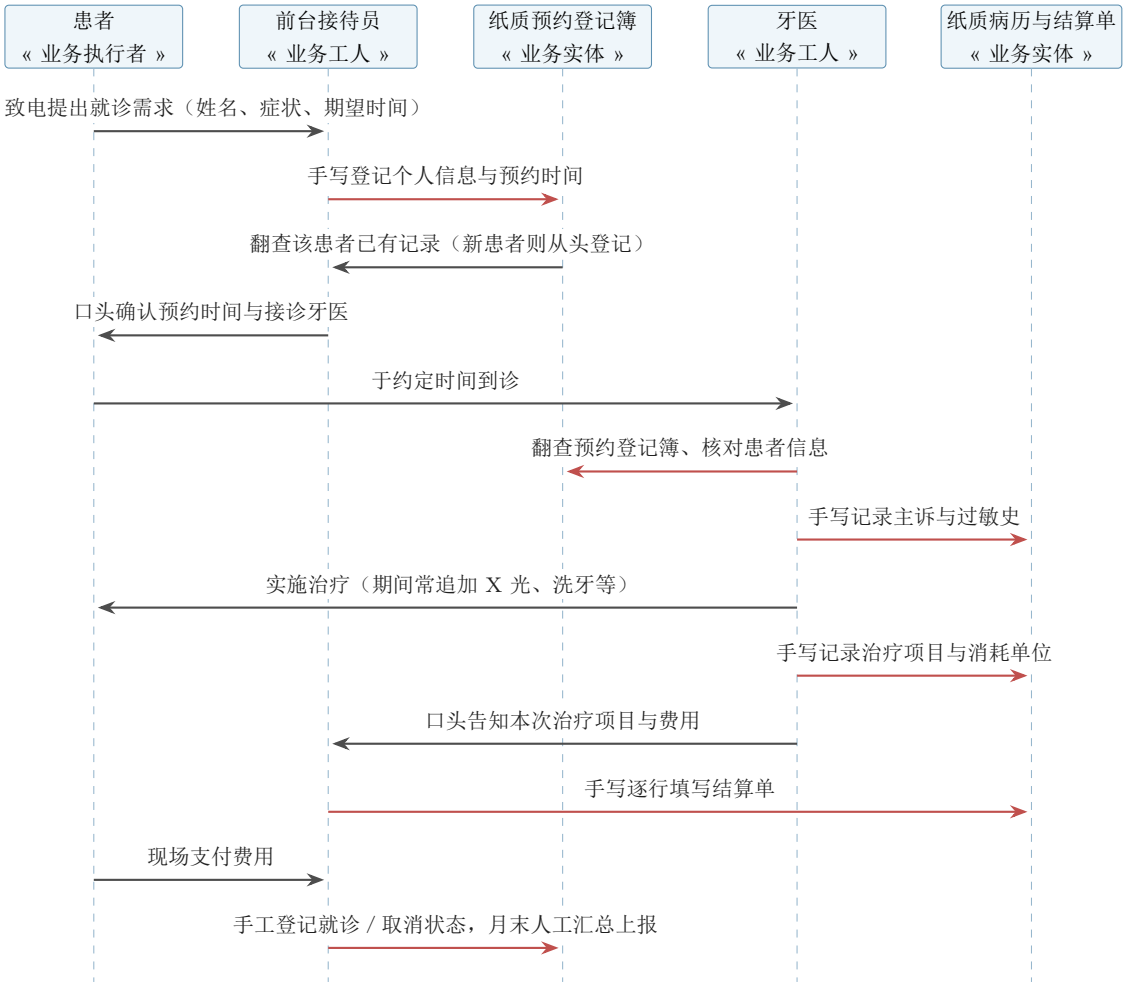


图 2　现状业务序列图：一次完整的预约与就诊

图 2 说明 这张图是“拿着摄像机去拍”拍到的真实情况，此时还没有拟开发系统。三条真相一目了然：

1. **信息在纸面上重复流转。**挂号信息、主诉、过敏史、治疗项目、费用，分别由前台和牙医手写在不同纸质表单上；同一位患者的信息要登记、查找、抄录多次，既慢又容易出错。

2. **就诊状态没有可靠的落点。**“这位患者到底来没来”只被手写在一本纸质登记簿上，月末再由人逐页统计后上报总部——取消率无法实时掌握，更谈不上按诊所、按牙医拆解。
3. **患者与牙医的匹配靠人的经验。**电话里安排哪位牙医接诊，取决于前台对各位牙医时间与专长的印象；牙医在多家诊所轮转，其资质与忙闲分散在不同诊所的台账里，总部无法形成统一视图。

3.4 改进后业务序列图

改进的思路是引入一个新的业务实体——**ADC 牙科连锁信息管理系统**，由它接手原来由纸质表单、各诊所本地台账和人工汇总承担的责任。改进点严格对应三条经典改进模式：物流变信息流、改善信息流转、封装领域逻辑。

改进后的业务序列图就是**业务蓝图**：它描述的不是现状，而是“组织将来要按这个样子运转”。因此在实际项目中，这一版必须经甲方确认——ADC 的首席执行官与各诊所主管签字认可后，才能作为需求提取的依据。本报告按案例事实绘制，签字环节在课程作业中省略；但这一步在真实项目里不能省，否则后面所有的用例、类图与表结构都建立在未经确认的假设上，改一处就要全线返工。

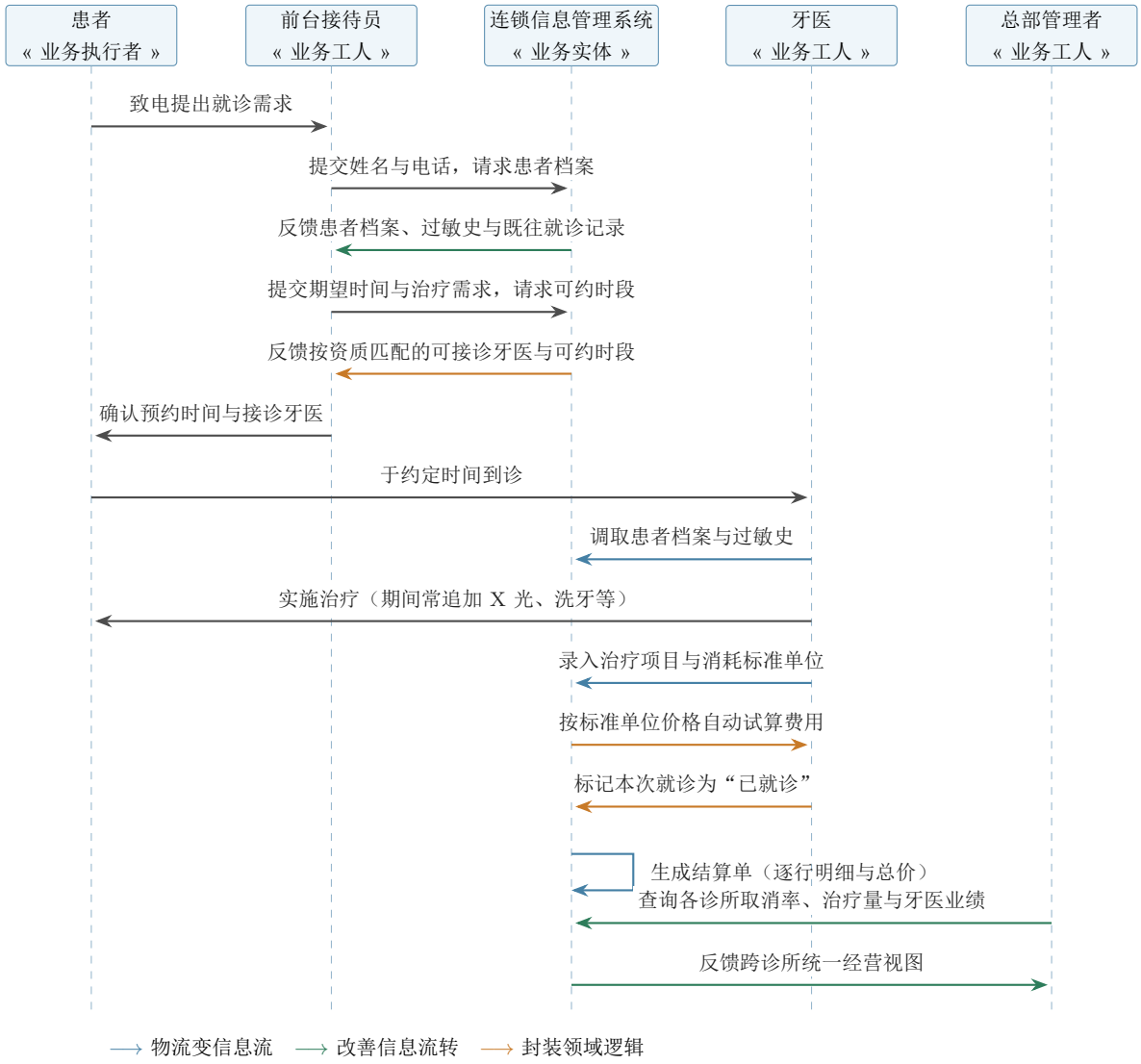


图 3　改进后业务序列图：引入“连锁信息管理系统”后的预约与就诊

图 3 说明 对照图 2，逐条看变化：

- **物流变信息流（蓝色）**：纸质病历、手写记录被“调取档案”“录入治疗项目”两条消息取代。病历不再跟着纸张走，而是作为记录直接进入系统；结算单也不再手工逐行填写，而是由本次就诊的治疗数据自动生成。
- **改善信息流转（绿色）**：前台不再翻纸质登记簿核对，一次查询即可调出跨诊所共享的患者档案与过敏史；总部管理者也不必再等各诊所月末上报，随时可查询各诊所的取消率与治疗量。数据从“各店各表”变成“全连锁一份”。
- **封装领域逻辑（橙色）**：原来锁在前台与牙医经验里的规则被系统封装——“按资质与时段的牙医匹配”“按标准单位与单价计价”“到诊 / 取消的自动标记”，都在系统中完成。前台与牙医从“靠记忆的人”变成“会用系统的操作者”，新诊所也能立即获得一致的业务口径。

同时注意：诊室里的实际治疗动作（“实施治疗”）仍然由牙医完成，系统只是接管了信息与规则的流转——这正是“系统替换的是记录与协调，而不是医术本身”的边界所在。

3.5 改进前后对照

表 1　业务环节改进前后对照

业务环节	现状	改进后	改进模式
患者档案	各诊所以纸质表登记，换店就诊须重新登记	全连锁统一档案，一店登记全连锁可见	改善信息流转
预约与排班	前台凭电话记录与印象手工安排	系统按牙医资质与可约时段匹配	封装领域逻辑
病历与治疗记录	手写纸质病历，治疗项目靠人工记	电子病历，治疗项目按标准单位录入	物流变信息流
计价与结算	手写结算单，价格靠记忆与查表	系统按标准单位价格生成逐行明细与总价	封装领域逻辑
就诊状态跟踪	纸质登记，月末人工统计，取消率不可算	每次预约为一条记录，到诊 / 取消自动标记	改善信息流转
牙医业绩与激励	无系统记录，10% 激励制度无法落地	按季度自动汇总收入、成本与运营费用	封装领域逻辑

4. 需求

需求从改进后业务序列图**映射**而来：与拟开发系统有实线相连的实例所代表的系统，成为系统执行者；从外部指向系统的消息，成为系统用例。但在映射之前，先要回答一个更基本的问题：这些需求凭什么算数？答案是它们来自涉众，并且经过取舍。

4.1 需求获取：五种方法与本案的取舍

需求不是从业务模型里推导出来的，而是从涉众那里取来的。课程给出五种获取方法，各自的适用面与局限如下表。

表 2　五种需求获取方法的比较

方法	做法	优点	局限	适用场景
研究资料	读案例、制度文件、报表与既有表单	成本低、可反复核对，术语与背景统一得快	二手材料，真实性不足；可能过时或带立场	项目启动、建立业务背景与术语表
实地调研（访谈）	到诊所访谈前台、牙医、主管，跟一遍流程	能追问例外情况，拿到“实际怎么做”	受访者易说“应该怎样”，样本小、有主观性	流程细节、痛点确认

方法	做法	优点	局限	适用场景
研究竞争对手	看同类连锁的公开资料、产品演示与招投标文件	对标行业惯例，补全功能清单	只能看到对外呈现，拿不到内部规则	功能范围校对
问卷调查	面向患者或员工发放结构化问卷	样本大、可量化、便于统计	设计门槛高、回收率低、无法追问	满意度、偏好、频次类问题
现场操作（跟岗）	亲自做一遍接电话、登记、开单	最接近真实，能发现“说不出口”的动作	耗时、需授权，观察者会影响被观察者	高频核心流程

本项目的特殊之处在于：目标组织是案例中的企业，无法进入现场。因此本报告以案例文本为主要证据来源，属于上表第一种方法；凡是案例写明的（组织规模、角色分工、取消率高、激励基金按净利润 10% 分配、患者档案跨店共享）直接采信，凡是案例没写到的细节（结算单字段、资质有效期、结算单号规则）按牙科行业通行做法补足，并在正文中标明为合理假设。这正是研究资料法的典型局限——方便、但真实描述不足——对策是把“案例事实”与“推断”分开写，推断处给出理由，便于评审时质疑和修正。若本项目有实地条件，访谈与跟岗应当优先补上，尤其是前台接电话这一段，因为本报告的业务序列图正是以它为骨架。

4.2 需求是涉众利益的平衡

需求文档不是把所有涉众的要求都收进去，而是在冲突处做出可解释的取舍。本案有三处真实冲突：

- **“快”与“全”的冲突。**患者希望一通电话说清，牙医不希望录入占用诊椅时间，总部却要字段齐全才能统计。取舍是：统计所需的数据由系统自动派生（就诊状态、金额、时长），不要求人多填一遍；前台必填项压到最少——姓名、电话、时间、牙医，其余选填。
- **“能改”与“不能改”的冲突。**前台希望随时能改时间、改价格、改状态；总部希望口径统一、事后可审计。取舍是：改期允许（走取消后重约），改价与改状态不允许直接编辑，只能经授权或走红冲留痕。
- **“灵活”与“统一”的冲突。**牙医希望按病例灵活定价，总部希望全连锁统一价目。取舍是：默认取标准治疗目录的单位价格，特殊调整须诊所主管授权并留痕，报表中单列调整金额。

这三处取舍分别落在 UC-02 的字段列表与业务规则、UC-03 的定价规则、UC-04 的红冲规则上，并在第六章的数据库约束里再兜一道底。规约中的“涉众利益”不是客套话，它记录的正是冲突各方的诉求；“业务规则”则是冲突的裁决结果。评审时如果只看到规则、看不到冲突，规则就成了无源之水；反过来，只罗列利益而不给裁决，需求就无法实现。

4.3 系统执行者

- **前台接待员（主执行者）**：主动发起登记患者档案、受理预约等用例。
- **牙医（主执行者）**：主动发起记录就诊与治疗、生成结算单、记录就诊状态等用例。
- **总部管理者（主执行者）**：维护牙医资质与诊所信息，查询统计与报表。
- **时间（主执行者）**：作为特殊的业务实体，驱动“统计牙医季度业绩”这一用例——注意它是“到季度末触发”，不等于系统里的定时器。

注意：患者只与前台、牙医发生业务交互，并不直接操作本系统（由前台代理完成），因此**不把患者列为系统执行者**；支付 / 保险机构在结算环节由系统主动请求，作为辅执行者写入“生成结算单”用例的设计约束，不再单列。这也为未来面向患者的自助预约留下了扩展空间。

4.4 系统用例图

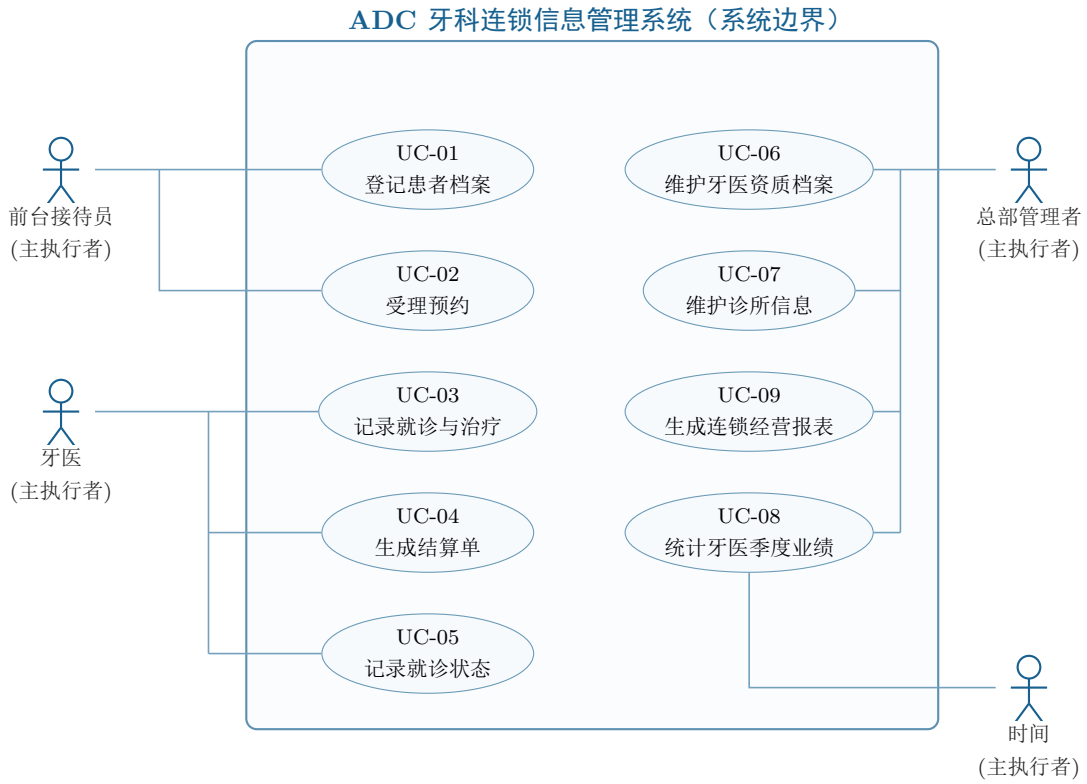


图 4　系统用例图：ADC 牙科连锁信息管理系统

图 4 说明 图中共 9 个系统用例。左侧两个执行者就是图 3 中的两类业务工人——业务工人“前台接待员”“牙医”在需求层面被映射为系统执行者，因为他们就是与系统直接交互的外部实例；右侧的总部管理者来自图 3 的“总部管理者”生命线。注意“时间”作为主执行者驱动 UC-08：它提供的是“到季度末汇总业绩”的价值，表示的是业务上的时间节点，而不是系统内部的定时器组件。UC-02、UC-03、UC-04 构成本系统的核心价值链（预约 → 治疗 → 结算），下一节给出完整规约。

4.5 系统用例清单

表 3　系统用例清单

编号	用例	主执行者	简述	优先级
UC-01	登记患者档案	前台接待员	记录姓名、出生日期、账单地址、电话、邮箱与紧急联系人，并逐项登记过敏症	高
UC-02	受理预约 ★	前台接待员	按牙医资质与可约时段匹配，生成预约并记录就诊备注	高
UC-03	记录就诊与治疗 ★	牙医	记录主诉与过敏史，录入治疗项目、消耗标准单位与单位价格	高
UC-04	生成结算单 ★	牙医	按标准单位价格生成逐行明细与小计，出具结算单与总价	高
UC-05	记录就诊状态	牙医	将预约标记为已就诊或未到诊，支撑取消率统计	中
UC-06	维护牙医资质档案	总部管理者	登记牙医个人信息、执业资质及其在多家诊所的轮转	中
UC-07	维护诊所信息	总部管理者	登记诊所地址、主管姓名、电话、邮箱与办公面积	中
UC-08	统计牙医季度业绩	总部管理者、时间	按季度汇总每位牙医的收入、成本与运营费用，支撑激励基金分配	中
UC-09	生成连锁经营报表	总部管理者	按诊所与时段输出取消率、治疗量与收入报表	低

注：标 ★ 为本系统的核心用例，下一节给出完整用例规约。

4.6 用例规约

以下三份规约分别对应价值链的入口（受理预约）、核心业务（记录就诊与治疗）、以及价值兑现（生成结算单）。规约写法遵循“涉及界面组件的细节不进需求”的原则：主语只用执行者名称或“系统”。

UC-02　受理预约

执行者	前台接待员（主执行者）
前置条件	前台接待员已登录系统并选定了接诊诊所；患者已通过电话提出就诊需求
后置条件	系统保存了一条预约记录（含患者、诊所、日期、时间、接诊牙医与就诊备注），状态为“待就诊”；若为新患者，其档案与过敏症已完成登记

涉众利益

- 患者——希望尽快约到合适的时间与牙医，希望过敏史与就诊需求被准确记录，不希望白跑一趟。
- 前台接待员——希望在一次通话内完成档案查询、牙医匹配与预约登记，不用在多个表格间来回切换。
- 牙医——希望预约信息准确（时间、治疗需求、过敏史），不希望被安排在自己不具备资质的治疗上。
- 诊所主管——希望诊室与牙医时间被合理占用，不出现空档或撞车。
- 总部管理者——希望每一次预约都进入统一记录，从而能够按诊所统计取消率与治疗量。
- 老板（老大）——希望预约口径在全连锁统一，使扩张后新增诊所也能即刻对齐管理标准。

基本路径

1. 前台接待员提交患者的姓名与电话号码，请求调取患者档案；
2. 系统反馈患者档案，包括既往就诊记录与过敏症清单；
3. 前台接待员提交期望就诊的日期、时间与治疗需求；
4. 系统反馈满足资质要求且该时段可接诊的牙医列表及其所在诊所；
5. 前台接待员选定牙医与时段，提交预约；
6. 系统校验该牙医时段未被占用、该诊所该时段可用；
7. 系统生成预约记录（状态为“待就诊”）；
8. 前台接待员向患者复述预约时间、诊所地址与接诊牙医，通话结束。

扩展

- **1a. 患者档案不存在（新患者）：** 1a1 系统提示该患者为新患者；1a2 前台接待员执行“登记患者档案”用例（UC-01）；1a3 返回基本路径 2。
- **2a. 患者有已知过敏症：** 2a1 系统高亮显示过敏源清单；2a2 前台接待员向患者确认过敏源是否有变化；2a3 返回基本路径 3。
- **3a. 患者就诊需求超出 ADC 现有资质范围：** 3a1 系统提示无匹配资质；3a2 前台接待员说明情况并建议转诊。
- **4a. 无满足条件的牙医可接诊：** 4a1 系统反馈邻近时段或邻近诊所的可选方案；4a2 前台接待员与患者协商调整时间或诊所；4a3 返回基本路径 3。
- **5a. 该牙医时段在提交瞬间已被他人占用：** 5a1 系统提示时段冲突并列出新的可约时段；5a2 返回基本路径 4。

字段列表

- 预约 = 预约编号 + 患者 + 诊所 + 就诊日期 + 就诊时间 + 接诊牙医 + {治疗需求}* + [就诊备注] + 预约状态 + 受理时间

- 患者档案 = 患者账号 + 姓名 + 出生日期 + 账单地址 + 电话号码 + [电子邮箱] + 紧急联系人姓名 + 紧急联系人电话 + {过敏症}*
- 过敏症 = 过敏源名称 + [严重程度] + [备注]
- 预约状态 = {待就诊 | 已就诊 | 未到诊 | 已取消}

业务规则

1. 预约编号在全连锁内唯一，规则为“诊所代码 + 日期 + 四位流水号”。
2. 每位牙医在同一时段（以 30 分钟为一个时段）最多只能有一个有效预约。
3. 同一患者的未到诊记录数达到约定阈值时，系统在受理新预约时给出提示，由前台接待员决定是否要求预缴费用。
4. 患者的过敏症必须逐项登记，不得只记“有过敏”；过敏症记录一经登记不得被后续就诊覆盖。
5. 牙医只能被安排在其持有相应资质的治疗需求上；无资质匹配时系统不给出该牙医。
6. 预约状态只由“记录就诊状态”用例（UC-05）从“待就诊”改为“已就诊”或“未到诊”。

质量需求

- 提交患者标识后，系统应在 2 秒内反馈患者档案（在连锁患者总数不超过 5 万人的条件下）。
- 支持 15 家诊所合计 120 名前台与牙医同时在线，不出现同一牙医时段被重复预约的数据冲突。
- 一名熟练前台接待员完成一次标准预约受理的人机交互不超过 6 次。

设计约束

- 患者档案在全连锁范围内共享，同一患者在不同诊所就诊不得产生重复档案，以患者账号或电话号码为识别键。
- 系统须支持新增诊所快速接入，新诊所开业即可使用与成熟诊所一致的预约口径。

UC-03　记录就诊与治疗

执行者	牙医（主执行者）
前置条件	患者已按预约到诊；该患者存在状态为“待就诊”的当日预约
后置条件	系统保存了一条就诊记录（含主诉、就诊备注与本次就诊的全部治疗行）；每个治疗行记录了消耗的标准单位与单位价格；预约状态已更新为“已就诊”

涉众利益

- 患者——希望治疗过程被完整准确记录，希望每项治疗的单位与价格可核对，不愿被多计费。
- 牙医——希望治疗项目与单位的录入尽量简短，不占用诊椅时间，也不希望事后补记遗漏。
- 前台接待员——希望结算时能直接取到治疗数据，不用再向牙医口头确认项目与金额。
- 总部管理者——希望各诊所采用统一的项目与单位口径，使各地治疗标准可比较。
- 老板（老大）——希望借助统一口径的治疗数据判断各地治疗标准是否一致，并跟踪牙医产能。

基本路径

1. 牙医提交当日预约列表或患者标识，请求调取就诊信息；
2. 系统反馈该患者的档案、过敏症清单与既往治疗记录；
3. 牙医录入主诉与就诊备注；

4. 牙医实施治疗；
5. 牙医为本次就诊录入选定的治疗项目及其消耗的标准单位与单位价格；
6. 系统校验所选治疗项目属于标准治疗目录；
7. 牙医确认本次就诊的治疗清单；
8. 系统保存就诊记录；该笔预约的状态由“记录就诊状态”用例（UC-05）统一改为“已就诊”，以保证预约状态的唯一改写入口；
9. 系统提示可生成结算单，牙医据此执行“生成结算单”用例（UC-04）。

扩展

- **2a. 患者提出新的症状或需求：** 2a1 牙医在就诊备注中追加记录；2a2 返回基本路径 3。
- **3a. 患者过敏症与拟施治疗存在冲突：** 3a1 系统给出过敏冲突警示；3a2 牙医更换材料或治疗方式后再行记录。
- **5a. 就诊期间追加治疗（如 X 光、洗牙、抛光）：** 5a1 牙医向患者说明并取得同意；5a2 牙医在治疗清单中追加相应项目与单位；5a3 返回基本路径 6。
- **6a. 所选治疗项目不在标准治疗目录中：** 6a1 系统提示该项目未定义；6a2 牙医改选标准项目，或提交新增项目的申请由总部管理者审核。
- **8a. 患者中途终止治疗：** 8a1 牙医记录实际完成的治疗项目；8a2 系统按实际完成项目计算费用并返回基本路径 8。

字段列表

- 就诊记录 = 就诊编号 + 预约 + 患者 + 接诊牙医 + 诊所 + 就诊日期 + 主诉 + [就诊备注] + {治疗行}* + 就诊状态
- 治疗行 = 治疗项目 + 施用单位数量 + 单位价格 + [牙位 / 部位] + [执行牙医]
- 治疗项目 = 项目名称 + 类别 {检查 | 洁治 | 修复 | 种植 | 其他} + 标准单位定义 + 标准单位价格

业务规则

1. 一次就诊可包含一项或多项治疗；治疗行必须在牙医确认后才计入本次就诊。
2. 治疗行的单位价格默认取标准治疗目录中的标准单位价格；特殊情况下的调整必须留痕并经诊所主管授权。
3. 就诊记录一经生成，其治疗行不得物理删除；更正须以追加红冲记录的方式留痕。
4. 治疗项目的消耗单位必须采用该项目定义的标准单位，不得以“各”等模糊单位代替。
5. 患者过敏症在就诊界面必须可见；若治疗材料与过敏症冲突，牙医不得在未另行确认的情况下继续录入该项治疗。

质量需求

- 治疗行录入与校验在 1 秒内完成，牙医录入一项治疗的人机交互不超过 4 次。
- 就诊记录提交后立即可被结算环节读取，不出现“治疗已录入但结算取不到数据”的情况。
- 就诊记录永久保留，可供事后追溯与医疗纠纷处理。

设计约束

- 治疗项目与标准单位价格由总部统一维护，各诊所只能选用、不能自行定义。
- 系统须记录每次就诊的操作留痕（操作人、时间），以便追溯。

UC-04　生成结算单

执行者	牙医（主执行者）；支付 / 保险机构（辅执行者）
前置条件	该次就诊存在状态为“已就诊”的就诊记录，且治疗行已由牙医确认；诊所已维护结算单抬头与开票信息
后置条件	系统生成一张结算单（含全连锁唯一单号）；结算单包含每项治疗的逐行明细（治疗名称、施用单位数量、单位价格、小计）与总价，状态为“已出具”

涉众利益

- 患者——希望结算单逐项清楚、金额一次算对，并支持保险报销所需的明细。
- 牙医——希望不用手算、手写结算单，治疗数据能自动带出。
- 前台接待员——希望收费时能直接调用结算单，不用再核对纸质单据。
- 总部管理者——希望 15 家诊所采用统一的结算单格式与编号规则，便于汇总与财务稽核。
- 老板（老大）——希望标准化开票支撑收入统计与激励基金的计算。
- 支付 / 保险机构——希望交易信息完整、可追溯。

基本路径

1. 牙医为已完成就诊的患者请求生成结算单；
2. 系统读取该次就诊的治疗行，按“单位数量 × 标准单位价格”计算每行小计；
3. 系统汇总各行小计得到结算单总价，并生成全连锁唯一的结算单号；
4. 系统向牙医展示结算单的逐行明细与总价；
5. 牙医确认结算单；
6. 系统保存结算单（状态为“已出具”），并将治疗行与结算单建立关联；
7. 系统向患者出具结算单，并按需通过支付 / 保险机构接口发起收款或理赔；
8. 支付 / 保险机构反馈处理结果，系统更新结算单状态。

扩展

- **2a. 存在经授权调整过单位价格的治疗行：** 2a1 系统在明细中标注调整后价格与原价的差异；2a2 返回基本路径 3。
- **4a. 患者对某项治疗有异议：** 4a1 牙医核对就诊记录与治疗行；4a2 如确需更正，按业务规则生成红冲记录；4a3 系统重新计算并返回基本路径 3。
- **5a. 牙医发现明细有误：** 5a1 返回基本路径 1，重新读取治疗行。
- **7a. 患者要求改为保险结算：** 7a1 系统向保险机构提交理赔所需明细；7a2 保险机构反馈可报销金额与自付金额；7a3 系统据此拆分结算单并继续执行基本路径 8。
- **8a. 收款失败：** 8a1 系统将结算单置为“待支付”并反馈失败原因；8a2 前台接待员改用其他支付方式重试；8a3 返回基本路径 8。

字段列表

- 结算单 = 结算单号 + 就诊记录 + 患者 + 诊所 + 开单日期 + {结算行}* + 总价 + 结算单状态
- 结算行 = 治疗名称 + 施用单位数量 + 单位价格 + 小计
- 结算单状态 = {待出具 | 已出具 | 待支付 | 已结算}

业务规则

1. 结算单号在全连锁内唯一；每张结算单关联且仅关联一次就诊。
2. 结算单总价恒等于各结算行小计之和；结算行的单位价格来源于就诊记录的治疗行，不得手工另行输入。
3. 结算单经确认后不可直接修改；更正须通过红冲并重新开单的方式留痕。
4. 保险理赔金额与患者自付金额必须分别列示，不得合并为一笔。
5. 存在未结算结算单的患者再次就诊时，系统应在受理与就诊环节给出提示，由诊所主管决定是否放行。

质量需求

- 结算单生成在 1 秒内完成，且金额计算无舍入误差（以分为最小单位）。
- 结算单可按连锁、按诊所、按时段检索，支撑总部月度与季度汇总。
- 生成的结算单可打印或导出为 PDF，格式在各诊所保持一致。

设计约束

- 结算单格式与编号规则由总部统一规定，各诊所不得自行变更。
- 结算必须通过支付 / 保险机构接口完成，系统不得保存患者银行卡信息。
- 结算单须带保险理赔所需的项目编码，以便与保险机构对接。

5. 分析：从类图到序列图与状态机图

需求阶段回答的是“系统要为执行者做什么”，分析阶段回答的是“系统内部由哪些对象协作来完成”。本阶段交付三个工件：**分析类图**给出系统的静态结构，**分析序列图**把每个用例的责任摊到具体的类上，**分析状态机图**则把一个类在所有用例中的状态迁移一次讲全——三者分别对应全局静态、局部动态、零件动态三个视角。

按方法论，一个系统用例对应一张分析序列图。本报告画出价值链上的 UC-02 与 UC-03 两张完整图作为样例：前者覆盖“查询档案—按资质筛选—校验时段—写入”的典型链路，后者覆盖“建主记录—追加明细—计算小计—完成”的典型链路，两类结构合起来足以说明责任如何分配。其余 7 个用例的类间协作，在第 5.7 节的“系统用例、控制类与主要实体类的对应”表中逐条给出，不重复画图。完整做法应补齐 9 张，这是本报告在篇幅上的取舍，不是方法论的要求。

5.1 分析类图的三种构造型

分析类图由三种构造型（stereotype）构成，各自的职责与识别规则如下表。

表 4　分析类图的三种构造型及其识别规则

构造型	职责	识别规则	本案对应
边界类	系统的输入、输出与简单过滤	每一个有接口的外部系统映射一个边界类	柜面终端界面、总部报表界面、支付网关接口、保险理赔接口、患者通知接口
控制类	控制用例流程，为实体类分配责任	每一个系统用例映射一个控制类	9 个系统用例对应 9 个控制类，如“受理预约控制”
实体类	封装核心域的逻辑与数据，是本阶段重点	用例与实体类是多对多关系	患者、牙医、诊所、预约、就诊、治疗项目、就诊治疗明细、结算单、资质、过敏症等

本报告的落点是**实体类**：只有实体类才携带核心域的数据结构，第六章的数据库设计完全由它推导而来；边界类与控制类在同一张类图中用构造型标注体现，保证交付的是完整的分析类图，而非残缺的实体类堆砌。需要说明的是，分析类图按惯例只列**名称、属性与关系**，不列操作：这一阶段要确认的是“系统里有哪些概念、它们怎么关联”，操作属于职责分配的细化，留到设计阶段精化业务层时给出（见后文“核心领域操作与责任分配”表）。这不是遗漏，而是两个阶段关注点不同。

类与属性的来源也要说清楚：类的名称来自用例规约里的**名词性概念**与**关键事件**（预约、就诊、结算单都是事件被记录下来的结果），属性来自规约中的**字段列表**，关系来自**业务规则**。三者各有一条可追溯的来路，类图才不是凭感觉画出来的。

5.2 实体类的命名自检

实体类应当来自领域中的真实概念，而不是把“某某信息、某某记录、某某数据表”这类人造词当类。下表是本报告逐项自检的结果。

表 5　实体类命名自检：从“不当做法”到“本案做法”

不当做法（应避免）	本案的做法	理由
在类名后加“类”	直接叫患者、就诊、诊所	类是建模元素，名称里不需要再重复说明
加“信息 / 记录 / 数据 / 表 / 库”等后缀	患者信息 → 患者；就诊记录 → 就诊；诊所数据 → 诊所	这些后缀是人造出来的包装，不是领域概念本身
给类加“标识属性”	标识由主键“患者编号”承担	标识是存储层的关注点，不进入概念属性
加一个笼统名为“状态”的属性	就诊. 就诊状态，取值明确为“已预约 / 已就诊 / 已取消”，并另设“是否取消”布尔标记	状态必须落到具体取值，否则模型无法验证，取消率也无从统计
把设计细节塞进类	只保留用例规约中出现过的概念	类图描述问题域，不描述实现手段

需要单独说明的是**“结算单”**：它不属于被禁止的名词后缀，而是牙科业务中真实存在的票据名称（与“发票”同类），案例原文也以“标准化开单”作为管理诉求，因此保留该命名。

5.3 实体类清单与概念属性

实体类的属性来自用例规约中的“字段列表”，抽取原则是：只保留用例中被真正读写的领域概念，不引入实现手段。下表共识别出 16 个实体类。

表 6　实体类清单及其概念属性

实体类	概念属性	主要来源用例
患者 Patient	患者编号、姓名、性别、出生日期、联系电话、电子邮箱、通讯地址、建档日期、备注	UC-01、UC-02、UC-03
过敏症 Allergen	过敏症编号、过敏症名称、类别、说明	UC-03（过敏史核对）
患者过敏记录 PatientAllergy	记录编号、严重程度、反应描述、首次记录日期	UC-03
员工 Employee	员工编号、姓名、性别、联系电话、入职日期、在职状态	UC-02、UC-06、UC-08
牙医 Dentist	执业证书编号、专业方向、职称	UC-02、UC-03、UC-08
前台接待员 Receptionist	岗位等级、可受理的业务范围	UC-01、UC-02、UC-04

实体类	概念属性	主要来源用例
总部管理者 Administrator	职务名称、分管范围	UC-06、UC-07、UC-08、UC-09
诊所 Clinic	诊所编号、诊所名称、所在城市、详细地址、联系电话、开业日期、诊椅数量、营业状态	UC-02、UC-07、UC-09
牙医执业点 DentistClinic	执业点编号、排班说明、是否主要执业点	UC-02、UC-06
资质 Qualification	资质编号、资质名称、颁发机构、有效期至	UC-06
牙医资质 DentistQualification	资质记录编号、取得日期、证书编号	UC-06、UC-08
预约 Appointment	预约编号、预约时间、预计时长、预约渠道、预约状态、登记时间、取消原因	UC-02、UC-05
就诊 Visit	就诊编号、到诊时间、结束时间、主诉、诊断结论、就诊状态、是否取消、记录时间	UC-03、UC-05
治疗项目 TreatmentItem	治疗项目编号、项目名称、标准单位、单位价格、是否启用、项目说明	UC-03、UC-09
就诊治疗明细 VisitTreatment	明细编号、执行数量、消耗标准单位、单项小计、执行时间	UC-03、UC-04
结算单 Statement	结算单编号、开单日期、治疗费用、材料费用、折扣金额、应付总额、支付方式、支付状态、支付时间、保险理赔编号	UC-04

5.4 类之间的关系与多重性

识别类间关系有固定顺序：**先找泛化，再找关联，最后判断聚合与组合**。先泛化，是因为它决定属性的归属——如果先做关联，同一份公共属性会被重复抄进每个子类，后面再抽出来就要动整张图；关联其次，是因为多重性一旦确定，外键落点随之确定；组合放在最后，是因为它要先判断“部分能不能离开整体活着”，而这依赖前两步已经理清的语义。

多重性是本章最关键的一步：它决定第六章里外键落在哪张表上。下表中凡标“0..*”的一方，都是将来承载外键的一方。

表 7　类之间的关系与多重性

关系	多重性	依据与说明
员工 → 牙医（泛化）	1 : 1	牙医是一名员工，共享姓名、联系电话等共同属性，只特化出执业特征

关系	多重性	依据与说明
员工 → 前台接待员（泛化）	1 : 1	前台接待员同属员工，特化出岗位等级
员工 → 总部管理者（泛化）	1 : 1	总部管理者同属员工，特化出分管范围
牙医 → 牙医执业点	1 : 0..*	案例中牙医可在多家诊所轮岗，故一人可有多个执业点
诊所 → 牙医执业点	1 : 0..*	一家诊所可接纳多名牙医，且同一牙医在不同诊所的排班不同
诊所 → 前台接待员	1 : 0..*	前台接待员归属某一家诊所
患者 → 预约	1 : 0..*	一名患者可多次预约
牙医 → 预约	1 : 0..*	一名牙医可被多次预约
诊所 → 预约	1 : 0..*	预约发生在具体诊所
预约 → 就诊	1 : 0..1	预约到诊后转化为一次就诊；爽约的预约不产生就诊，这正是取消率的统计口径
患者 → 就诊	1 : 0..*	一名患者可多次就诊
牙医 → 就诊	1 : 0..*	一次就诊由一位牙医负责
诊所 → 就诊	1 : 0..*	就诊发生在具体诊所
就诊 → 就诊治疗明细	1 : 1..*	组合关系：明细随就诊产生、随就诊消亡，一次就诊至少执行一项治疗
治疗项目 → 就诊治疗明细	1 : 0..*	同一治疗项目可被反复执行，项目本身独立于就诊存在
就诊 → 结算单	1 : 0..1	一次就诊最多开出一张结算单
患者 → 患者过敏记录	1 : 0..*	一名患者可登记多项过敏症（案例强调过敏史必须逐项可查）
过敏症 → 患者过敏记录	1 : 0..*	同一过敏症可发生在多名患者身上
牙医 → 牙医资质	1 : 0..*	一名牙医可持有多项资质证书
资质 → 牙医资质	1 : 0..*	同一资质可由多名牙医持有

上表中有四处是**多对多**关联（牙医与诊所、患者与过敏症、牙医与资质、就诊与治疗项目）。按课件的映射规则，多对多不能直接落成外键，必须引入关联表——它们分别对应第六章的牙医执业点、患者过敏记录、牙医资质、就诊治疗明细四张表。

5.5 分析类图（一）：患者与诊疗域

16 个实体类按域拆成两幅图，以便看清结构与连线。图 5 是患者与诊疗域。

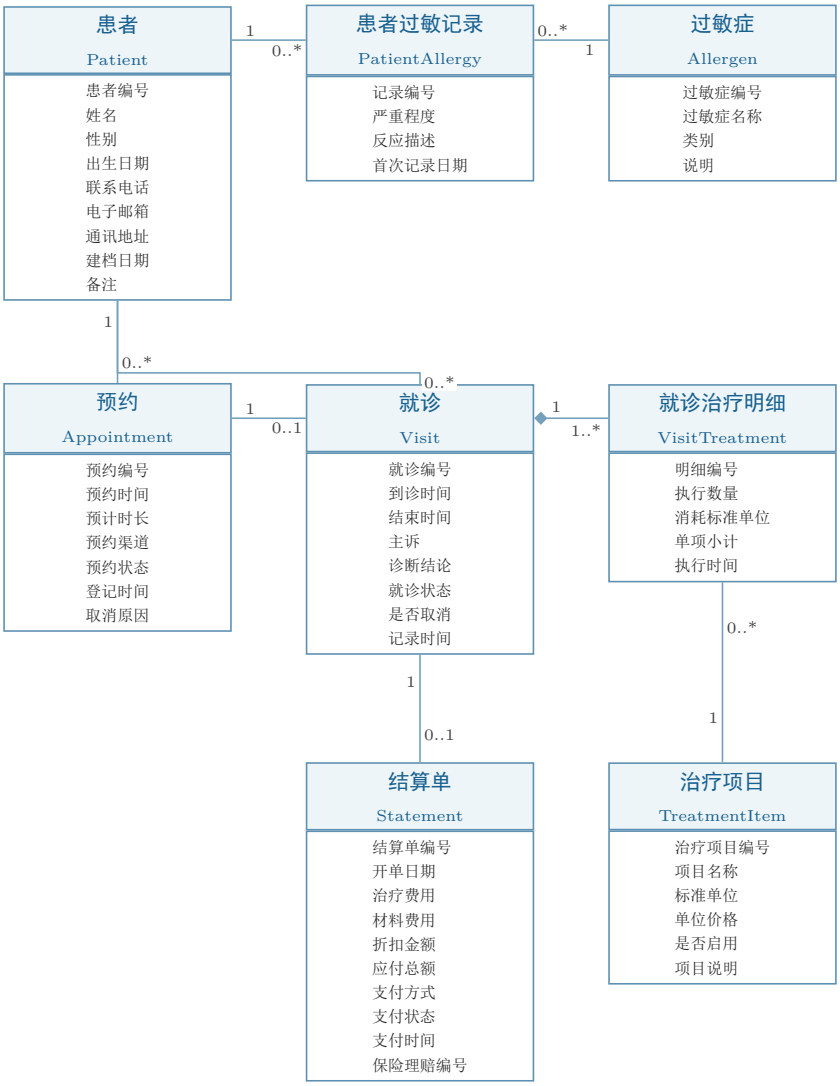


图 5　分析类图（患者与诊疗域）

图 5 说明　八个实体类覆盖了从建档、预约、就诊到结算的完整链条。三处结构值得注意：其一，**患者与过敏症是多对多**，因此中间必须有“患者过敏记录”这个关联类，且它自带严重程度、首次记录日期等属性——这正是案例“过敏史逐项可查”的落点；其二，**预约与就诊相互独立**，预约到诊后转化为就诊，爽约的预约不产生就诊，取消率因此有了统计口径，这是第二章愿景指标的结构支撑；其三，**就诊与就诊治疗明细是组合关系**（1 : 1..*），明细不能脱离就诊单独存在，而治疗项目是独立的字典类，与明细构成 1 : 0..*。此外，预约与就诊还各关联到牙医与诊所，该部分属于组织人员域，见图 6。

5.6 分析类图（二）：组织与人员域

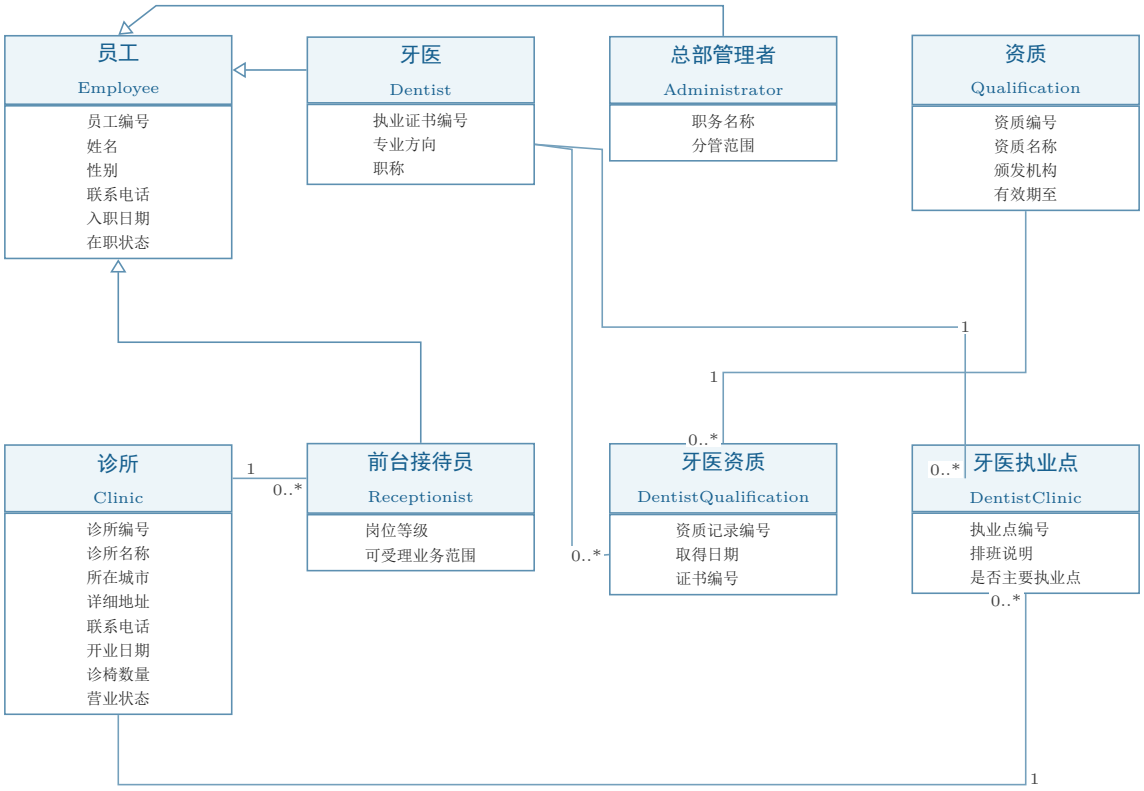


图 6　分析类图（组织与人员域）

图 6 说明　这幅图承担两件事。第一是**泛化**：牙医、前台接待员、总部管理者都是员工，姓名、联系电话、入职日期等共同属性上提到“员工”这一超类，三类人员只保留各自特化的属性——案例中“牙医与前台是两类不同的人、共享部分人事信息”正是这个结构。第二是**多对多的落点**：牙医可以在多家诊所轮岗，故牙医与诊所之间用“牙医执业点”作为关联类，它还携带排班说明这一业务属性；牙医可持有多项资质，故用“牙医资质”作为关联类记录取得日期与证书编号。这两处关联类，与前图的“患者过敏记录”一样，都是多对多关联在分析阶段的正式表达，第六章的关联表就是它们的直接投影。

5.7 分析类图（三）：边界类与控制类



图 7　分析类图（边界类、控制类与实体类的关系）

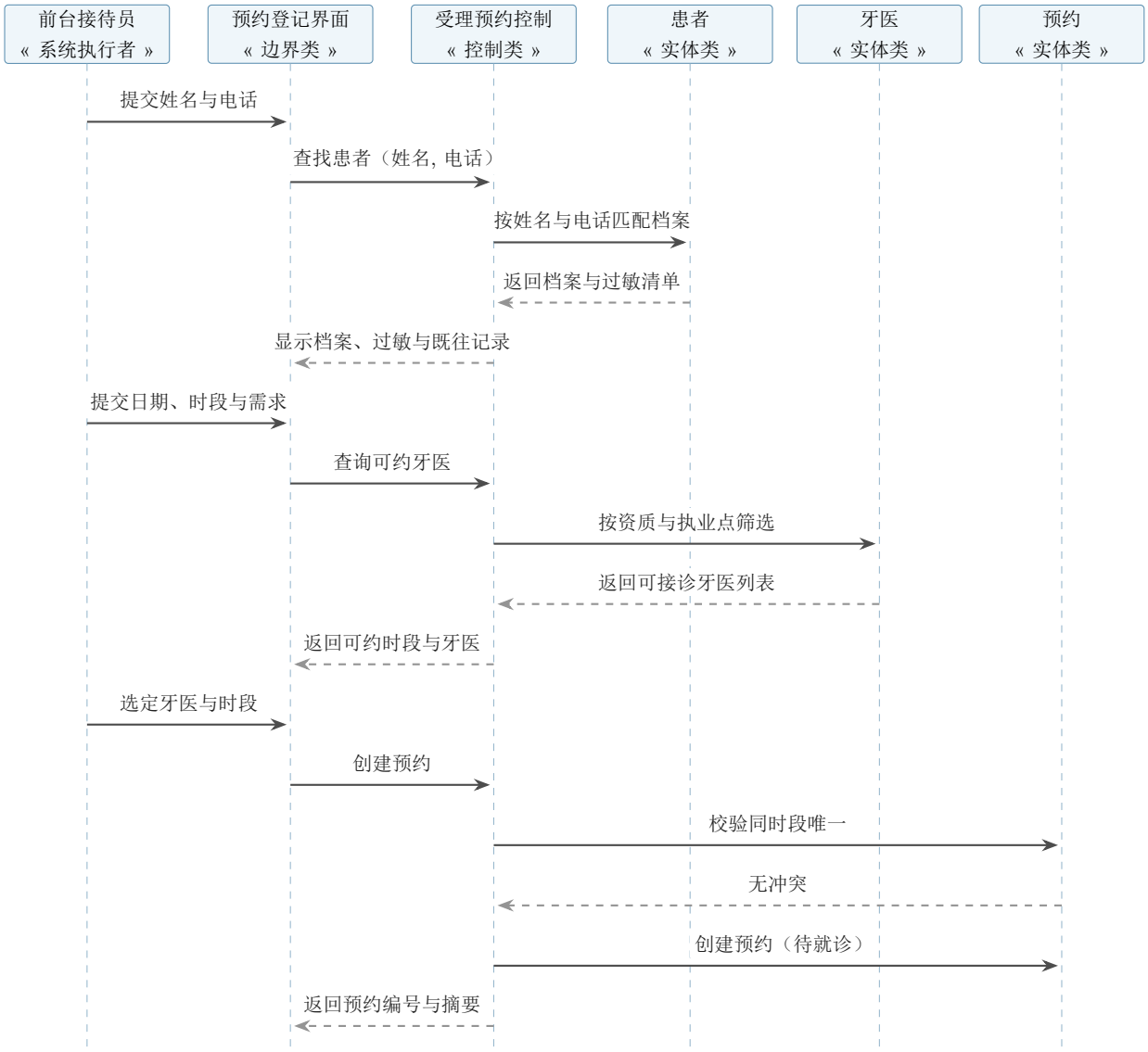
图 7 说明　边界类负责输入输出，是外部执行者与系统之间的隔离带——支付网关、保险理赔、患者通知三个接口类分别对应三个真实的外部系统，柜面终端界面与总部报表界面则对应两类人机交互入口。控制类按“一个系统用例一个控制类”的规则一一对应，由它编排流程、向实体类分配责任；实体类承担数据与核心域逻辑，图 5 与图 6 展开的就是它的内部结构。这条“边界 → 控制 → 实体”的链路，正是接着进入设计阶段的抓手。

表 8　系统用例、控制类与主要实体类的对应

用例	控制类	主要协作的实体类	说明
UC-01	登记患者档案控制	患者、患者过敏记录	建档时同步登记过敏史
UC-02	受理预约控制	预约、患者、牙医、诊所	按资质与执业诊所匹配可接诊牙医
UC-03	记录就诊与治疗控制	就诊、就诊治疗明细、治疗项目、过敏症	按标准单位记录消耗并试算费用
UC-04	生成结算单控制	结算单、就诊、就诊治疗明细	汇总明细生成标准化结算单
UC-05	记录就诊状态控制	就诊、预约	标记“已就诊 / 已取消”，供取消率统计
UC-06	维护牙医资质档案控制	牙医、资质、牙医资质	资质有效期是排班的匹配条件
UC-07	维护诊所信息控制	诊所、员工	新诊所开业即接入统一口径
UC-08	统计牙医季度业绩控制	牙医、就诊、结算单	支撑净利润 10% 激励基金核算
UC-09	生成连锁经营报表控制	诊所、就诊、结算单、治疗项目	跨诊所统一经营视图

5.8　分析序列图（一）：UC-02 受理预约

分析类图回答“系统里有什么”，分析序列图回答“这些类怎么协作”。它与第三章的业务序列图不是同一种东西：业务序列图的研究对象是组织内各系统（人肉系统与电脑系统），最小颗粒是系统；分析序列图的研究对象是系统内部的类，最小颗粒是软件类。它的作用是把用例规约里的每一条职责，精确地分配到边界类、控制类与实体类上——这正是从“系统做什么”过渡到“系统里有什么”的那一步。

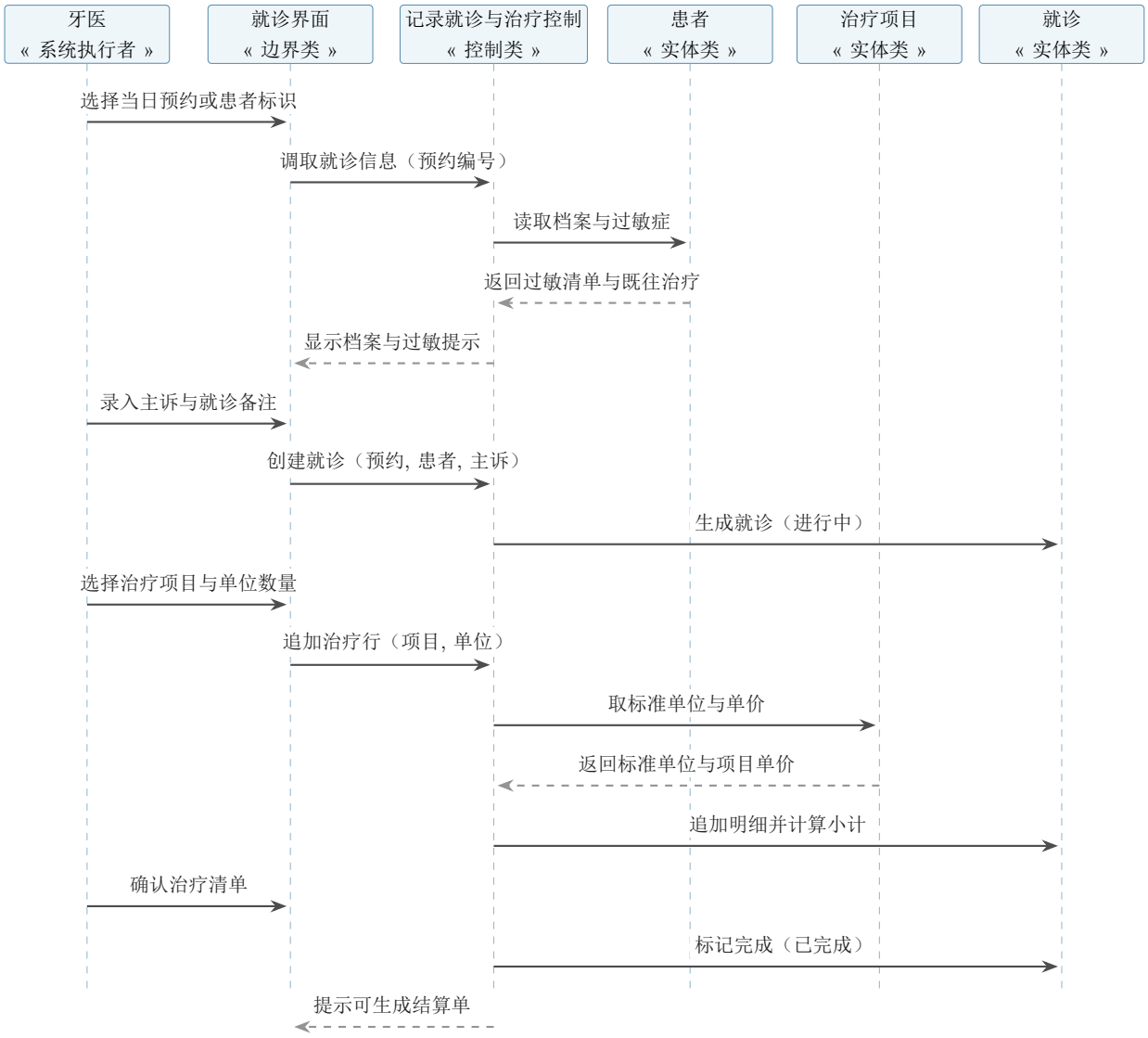


实线＝调用，虚线＝返回；生命线自左至右为执行者、边界类、控制类、实体类。

图 8　UC-02 受理预约的分析序列图

图 8 说明 图中第 1–5 条消息对应 UC-02 基本路径第 1–2 步，第 6–10 条对应第 3–4 步，第 11–16 条对应第 5–7 步。三点值得注意：其一，界面类只做输入采集与展示，患者匹配、牙医筛选、时段唯一性校验全部由控制类转交实体类完成，界面不承载任何业务判断；其二，**“校验同时段唯一”由预约实体自己负责**，而不是控制类里的一个 if 判断——这条规则属于预约这个概念本身，放在实体类里才能在“改期”“补录”等其它入口复用；其三，第 16 条返回的预约编号由实体生成后逐层回传，控制类不自行编造编号。

5.9 分析序列图（二）：UC-03 记录就诊与治疗



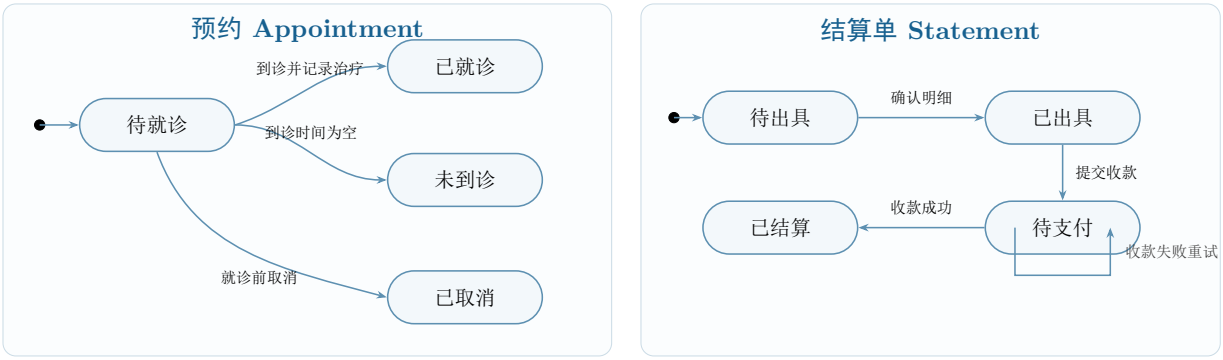
单价由治疗项目实体给出，不由界面传入；就诊状态由就诊实体自身维护。

图 9　UC-03 记录就诊与治疗的分析序列图

图 9 说明　第 1–5 条消息对应基本路径第 1–2 步，第 6–8 条对应第 3 步，第 9–13 条对应第 5–7 步，第 14–16 条对应第 8–9 步。这里体现两条设计判断：**单价只能来自治疗项目实体**（第 11–12 条），界面即使显示价格也只是只读回显，从根上堵住“前台改价”；**就诊状态由就诊实体自己迁移**（进行中 → 已完成，第 15 条），控制类只下达“完成”这个意图，不直接改字段。治疗行一旦确认即不可物理删除，图 9 的追加语义与 UC-03 的“红冲留痕”规则一致。

5.10 分析状态机图：预约与结算单

类图是全局、静态的视角，序列图是局部（单个用例）、动态的视角，状态机图则是单个零件（类）、动态的视角——它把一个类在所有用例中可能经历的状态与迁移一次讲全。ADC 案例里状态最多、最容易被漏写的是**预约**与**结算单**。



初始状态。状态名与数据库状态字段取值一一对应；迁移只由实体类操作触发，界面不得直接赋状态。

图 10　预约与结算单的分析状态机图

图 10 说明 左侧四个状态与 UC-02 字段列表里的“预约状态”取值完全一致，右侧四个状态与 UC-04 字段列表里的“结算单状态”一致——状态机不是另起一套术语，而是把这些取值的合法迁移画出来。三条规则由此变得可执行：“未到诊”只能由“到诊时间为空且预约时段已过”触发；“已取消”只在就诊前允许；“已结算”是终态，要更正只能红冲并重新开单，不能把已结算改回待支付。

表 9　状态迁移与用例、操作的对应

类	当前状态	事件	守卫条件	目标状态	触发用例
预约	待就诊	就诊记录完成	存在对应的就诊记录	已就诊	UC-05（UC-03 结束后调用）
预约	待就诊	预约时段已过且无就诊记录	当前时间 > 预约时间 + 宽限	未到诊	UC-05（按日批量）
预约	待就诊	患者或诊所取消	当前时间 < 预约时间	已取消	UC-02、UC-05
结算单	待出具	牙医确认治疗清单	明细非空且单价已确定	已出具	UC-04
结算单	已出具	提交收款	支付方式已选	待支付	UC-04
结算单	待支付	收款成功	支付网关返回成功	已结算	UC-04
结算单	待支付	收款失败	支付网关返回失败	待支付	UC-04（记录失败原因）

上表把图 10 的每条弧落到具体的用例与守卫条件上。守卫条件这一列是设计阶段要写进代码的判断，例如“当前时间 < 预约时间”才允许取消，正是案例里“取消率”统计口径能否成立的前提——如果允许事后把“未到诊”改成“已取消”，取消率就失去了意义。

6. 设计：数据层、业务层与表示层

分析类图描述的是对象世界，而拟开发系统最终要落在可运行的软件上。设计阶段按课程的三个动作推进：先**建立数据层**（把实体类图映射成表结构），再**精化业务层**（给类补上领域操作、把状态机落到代码），最后**精化表示层**（确定边界类承担的输入输出与窗体导航）。映射规则并非自创，而是课程给出的标准对应关系，本报告逐条落实。

6.1 类图到表的映射规则

表 10　对象概念与关系数据库概念的对应

对象概念	关系数据库概念	本案的落实
类	表	16 个实体类映射为 16 张表
属性	列	概念属性映射为列，不含操作
对象	行	一名患者、一次就诊各是一行
泛化	超类主键作为所有子类的主键	牙医、前台接待员、总部管理者三张表的主键都是 employee.employee_id
关联	外键放在**非 1 方**	例如患者与预约是 1 : 0..*，外键 patient_id 放进 appointment
组合	外键放在非 1 方，且**不为空**	就诊治疗明细的 visit_id 设为 NOT NULL，明细不能脱离就诊存在
多对多关联	独立的**关联表**（中间表）	四处多对多分别落成患者过敏记录、牙医执业点、牙医资质、就诊治疗明细四张表

主键遵循课件提出的两条要求：**唯一且恒定**，并且**把领域字段与主键分开**。因此本案所有表都采用代理主键（patient_id、clinic_id 这类编号），而不拿身份证号、邮箱、手机号等可能变化的领域字段充当主键。

6.2 泛化怎么落：三种做法与本案的选择

泛化是类图里最容易被顺手带过、却在落表时分歧最大的一处。课程要求比较“一张表”与“多张子类表”的优缺点，完整地说有三种做法：

表 11　泛化的三种落法及其取舍

做法	结构	优点	缺点	适用
单表继承	只建 employee 一张表，含“人员类型”列与三类人员的全部专属列	无连接，查任意人员都是一次查询；改类型只改一行	空列多；子类专属字段无法用 NOT NULL 约束；子类一多表就横着长	子类差异小、子类数量少、查询以“看全部人”为主
子类表（类表继承）	employee 存公共属性，dentist、receptionist、administrator 各存专属属性，子类主键即外键	专属字段可 NOT NULL；职责清晰；新增子类不动别的表	查全部人员要连接；插入要写两张表（同一事务）	子类属性差异大、子类身份本身就是业务规则
具体类表	不建超类表，每个子类一张完整表，公共属性各自重复	每个角色各自独立，单表查询最快	公共属性重复三份，无法用统一的员工编号关联；人员换岗要跨表搬数据	各角色几乎无共性、也不需要在同一视图里出现

本案选择**子类表**，理由有三条：其一，三类人员的专属属性差异很大（执业证书编号、专业方向、职称对岗位等级对分管范围），单表会留下大量永远为空的列；其二，泛化在这里不只是“代码复用”，它本身就是业务规则——牙医必须有执业证书，子类表可以把该列设为 NOT NULL，单表继承做不到这一点，只能靠应用层校验；其三，后续增加“牙科助理”这类新角色时，加一张表即可，不影响既有三张表。

代价也承认：查“全部员工名单”需要连接四张表，插入一名牙医要在一个事务里写 employee 与 dentist 两行。判断依据是频次——本案的报表以“按角色分别统计”为主（牙医业绩、前台排班），全员工混合列表几乎不出现，所以连接的代价小于空列的代价。反过来说，如果子类只多出一两个属性，或查询全是“看全部人员”，单表继承更划算。

6.3 自反关系与值表

自反关系是同一张表内部的外键，即一行的某一列指向本表的另一行。课程举的两个典型是会计科目表（科目有上级科目）与商品类别（类目树）。它的落法很固定：单表，加一列 parent_id 指向本表主键，不加新表；查询整棵树用递归查询或物化路径。判断要不要建自反结构，看层级是否需要事先固定——固定两三级用枚举列即可，层级不固定、随时可能加深才建自引用。

本案的模型里**没有自反结构**：治疗项目的分类用“检查 / 洁治 / 修复 / 种植 / 其他”这个枚举表达，只有一级；牙医与诊所是多对多，靠 dentist_clinic 关联表，不是自反；员工之间没有上下级汇报关系（诊所主管是岗位，不是员工的上级节点）。如果总部以后要求治疗项目做多级目录

（例如“修复 → 冠修复 → 全瓷冠”），改法是给 treatment_item 加一列 parent_item_id 自引用外键，而不是新建一张分类表——这也是自反关系最常见的用法：用一列换一张表。

值表（lookup table）是另一类容易被忽略的表：它行数少、变更慢、以“编号 + 名称 + 少量属性”为主，本身几乎不引用别的表，只被业务表引用。本案中 treatment_item（标准治疗目录）、qualification（资质字典）、allergen（过敏症字典）都属于这一类。值表的作用是把可变的编码与分类从业务表里抽出来：治疗项目调价只改 treatment_item 一行，历史结算单不动；新增一种资质只加一行，牙医资质表不用改结构。值表还应当**只停用、不删除**——treatment_item 上的 is_active 就是为此而设，因为历史就诊记录仍然引用着已停用的项目。

需要区分的是：clinic 虽然也被大量引用，但它是组织实体（有地址、开业日期、诊椅数量，会开业与关闭），不是值表；把组织实体当值表处理，会让“诊所关闭”这个业务动作无处安放。

6.4 数据访问方式：四种模式与本案的选择

从业务对象到数据库行，课程给出四种数据访问方式，粒度与耦合度依次不同：

表 12 四种数据访问方式的比较

模式	粒度	做法	优点	缺点
表入口	一张表	一张表一个类，方法直接对应查询、插入、更新、删除	简单直观，SQL 集中在一处	只有数据操作，领域逻辑无处安放
行入口	一行	一行一个对象，对象只带字段与增删改查，不含业务行为	比表入口更面向对象	对象是“数据袋子”，逻辑仍堆在服务层
数据对象（活动记录）	一行	一行一个对象，对象自己负责保存与更新	开发快、代码少，小系统够用	领域对象依赖数据库，难测试、难换库
数据映射器	一个领域对象	领域对象不认识数据库，由映射器负责对象与行之间的转换	领域模型纯净、可单测、可换库	类数量增加，需要 ORM 或手写映射

本案选择**数据映射器**，理由与前面的业务规则直接相关：其一，本系统的规则密度高（时段唯一、资质匹配、红冲留痕、状态只能由指定操作改写），这些规则必须放在领域对象里，而活动记录会让领域对象继承一个持久化基类，测试时脱离不了数据库；其二，质量需求要求“新增诊所快速接入、口径统一”，映射器把 SQL 与表名隔在一层，换库或分库时只替换实现；其三，它与下一节的领域驱动设计一致——**业务对象与数据访问分离**。

实现上遵循依赖倒置：映射器接口定义在业务层，MySQL 实现放在数据层，业务层只依赖接口（图 13 中“数据映射器 DentalMapper”那一格就是实现侧）。这样做的检验标准很朴素：把 MySQL 换成别的存储，业务层的代码一行不用改。

6.5 每条关联的外键落点

下表是“转化”这一步最直接的证据：第五章识别出的每一条关联，都在这里指定了外键落在哪张表的哪一列上。

表 13 每条关联的外键落点

关联	多重性	外键落点
员工 → 牙医（泛化）	1 : 1	dentist.employee_id 既是主键又是外键，指向 employee.employee_id
员工 → 前台接待员（泛化）	1 : 1	receptionist.employee_id 同为主键与外键
员工 → 总部管理者（泛化）	1 : 1	administrator.employee_id 同为主键与外键
诊所 → 前台接待员	1 : 0..*	receptionist.clinic_id ⇒ clinic.clinic_id
牙医 → 牙医执业点	1 : 0..*	dentist_clinic.employee_id ⇒ dentist.employee_id
诊所 → 牙医执业点	1 : 0..*	dentist_clinic.clinic_id ⇒ clinic.clinic_id
患者 → 预约	1 : 0..*	appointment.patient_id ⇒ patient.patient_id
牙医 → 预约	1 : 0..*	appointment.employee_id ⇒ dentist.employee_id
诊所 → 预约	1 : 0..*	appointment.clinic_id ⇒ clinic.clinic_id
预约 → 就诊	1 : 0..1	visit.appointment_id ⇒ appointment.appointment_id（加唯一约束）
患者 → 就诊	1 : 0..*	visit.patient_id ⇒ patient.patient_id
牙医 → 就诊	1 : 0..*	visit.employee_id ⇒ dentist.employee_id
诊所 → 就诊	1 : 0..*	visit.clinic_id ⇒ clinic.clinic_id
就诊 → 就诊治疗明细（组合）	1 : 1..*	visit_treatment.visit_id ⇒ visit.visit_id（NOT NULL）
治疗项目 → 就诊治疗明细	1 : 0..*	visit_treatment.item_id ⇒ treatment_item.item_id
就诊 → 结算单	1 : 0..1	statement.visit_id ⇒ visit.visit_id（加唯一约束）
患者 ↔ 过敏症（多对多）	多: 多	关联表 patient_allergy：patient_id、allergen_id 双外键，另带 severity、reaction、first_record_date
牙医 ↔ 诊所（多对多）	多: 多	关联表 dentist_clinic：employee_id、clinic_id 双外键，另带 schedule_note、is_primary
牙医 ↔ 资质（多对多）	多: 多	关联表 dentist_qualification：employee_id、qualification_id 双外键，另带 obtain_date、certificate_no

表中前三行的“超类主键作为子类主键”是泛化特有的映射方式，它保证了牙医、前台接待员、总部管理者不会出现与员工不一致的身份；后三行则说明：多对多关联一旦带上业务属性（严重程度、排班说明、取得日期），落成的关联表同时也就是第五章识别出的那个关联类——分析与设计在这里严丝合缝地对上了。

6.6 数据库表清单：16 张表的主键与外键

前两节讲的是“怎么映射”，这一节把映射结果一次列全。全系统共 **16 张表**，分为四组：**6 张基础表**（无外键）、**3 张人员子类表**（主键即外键）、**4 张关联表**（多对多的产物）、**3 张业务主表**（预约、就诊、结算单）。下面按两个领域分列，与图 11、图 12 一一对应。

表 14　组织与人员域 8 张表的主键与外键

序	表（中文 / 英文）	主键	外键（→ 被引用表）	存在理由
1	员工 employee	employee_id	无	三类人员共用
2	牙医 dentist	employee_id	employee_id → employee	执业证书与职称
3	前台接待员 receptionist	employee_id	employee_id → employee；clinic_id → clinic	所属诊所与范围
4	总部管理者 administrator	employee_id	employee_id → employee	职务与分管范围
5	诊所 clinic	clinic_id	无	门店档案与状态
6	牙医执业点 dentist_clinic	dc_id	employee_id → dentist；clinic_id → clinic	轮岗与排班
7	资质 qualification	qualification_id	无	资质目录
8	牙医资质 dentist_qualification	dq_id	employee_id → dentist；qualification_id → qualification	取证与有效期

患者与诊疗域是业务主线所在：预约、就诊、治疗明细、结算单四张表串起一次完整的诊疗过程，患者过敏记录与治疗项目则分别为它提供安全校验与计价依据。

表 15　患者与诊疗域 8 张表的主键与外键

序	表（中文 / 英文）	主键	外键（→ 被引用表）	存在理由
1	患者 patient	patient_id	无	患者主档
2	过敏症 allergen	allergen_id	无	过敏原目录
3	患者过敏记录 patient_allergy	record_id	patient_id → patient；allergen_id → allergen	严重程度与反应
4	预约 appointment	appointment_id	patient_id → patient；employee_id → dentist；clinic_id → clinic	取消率统计载体
5	就诊 visit	visit_id	appointment_id → appointment（唯一）；其余三个外键同第 4 行	到诊与诊断
6	治疗项目 treatment_item	item_id	无	单价目录
7	就诊治疗明细 visit_treatment	vt_id	visit_id → visit（NOT NULL）；item_id → treatment_item	组合关系落点
8	结算单 statement	statement_id	visit_id → visit（唯一）；patient_id → patient	财务凭证

两份清单可以直接当建库的核对表用，四条键规则可以一句话记住：

- **主键一律用代理键**，且都满足“唯一且恒定”；身份证号、手机号、邮箱这些会变的领域字段只作普通列，不承担主键职责。
- **泛化看人员三表：**牙医、前台接待员、总部管理者的主键与员工表是同名列、同取值，既是主键又是外键，所以一名牙医不可能挂在一个“不存在的员工编号”上。
- **关联看四张关联表：**牙医执业点、牙医资质、患者过敏记录、就诊治疗明细各持两条外键，并各自带上业务列（排班说明、取得日期、严重程度、执行数量）——多对多关系在这里既被拆开，又不丢语义。
- **一对一靠唯一约束：**预约与就诊、就诊与结算单都是 1 : 0..1，落成外键后在非 1 方加 UNIQUE，而不是让两张表互相引用。

这套结构只有 16 张表，却同时满足了三件事：任何一次取消都能按诊所、按牙医、按时间切片统计；任何一笔收费都能回溯到具体就诊、具体治疗项目与单价；任何一名牙医的资质都能在开单前被程序校验。

6.7 数据库图表（一）：组织与人员域

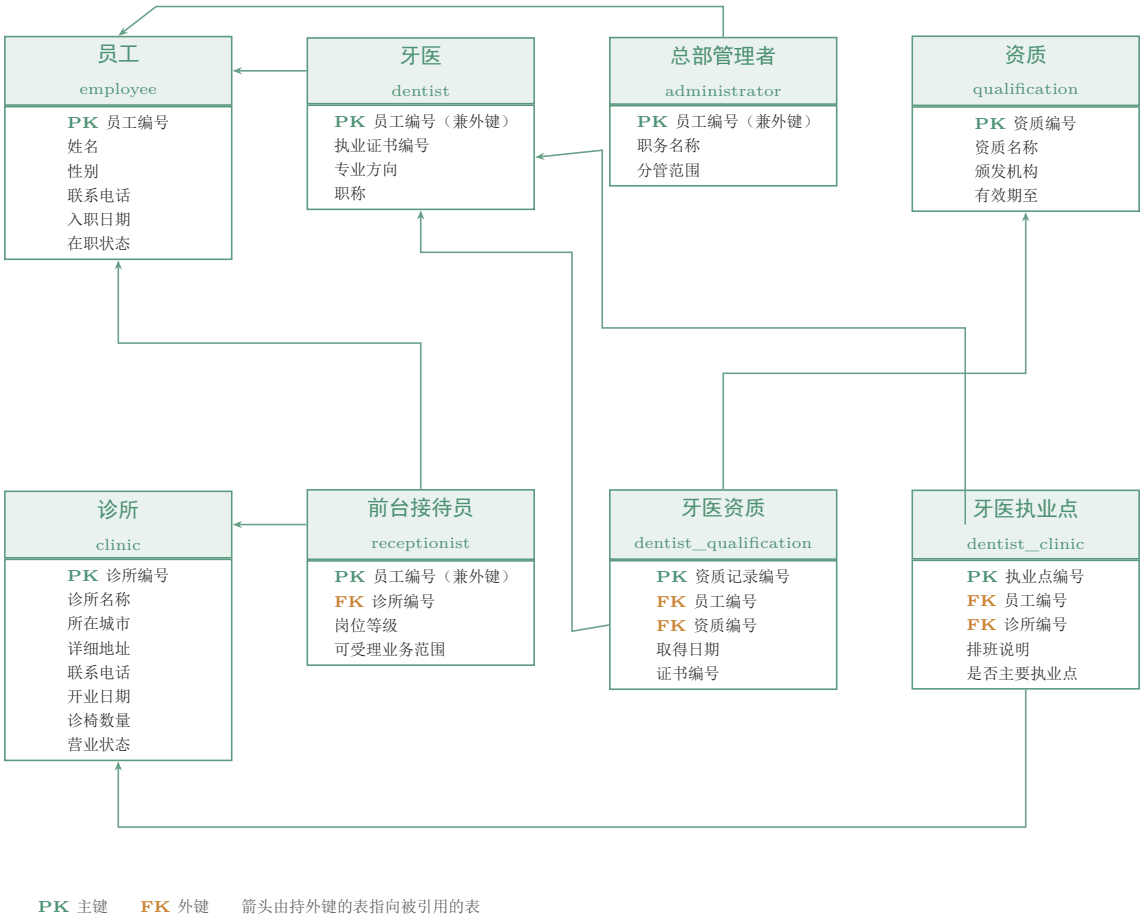
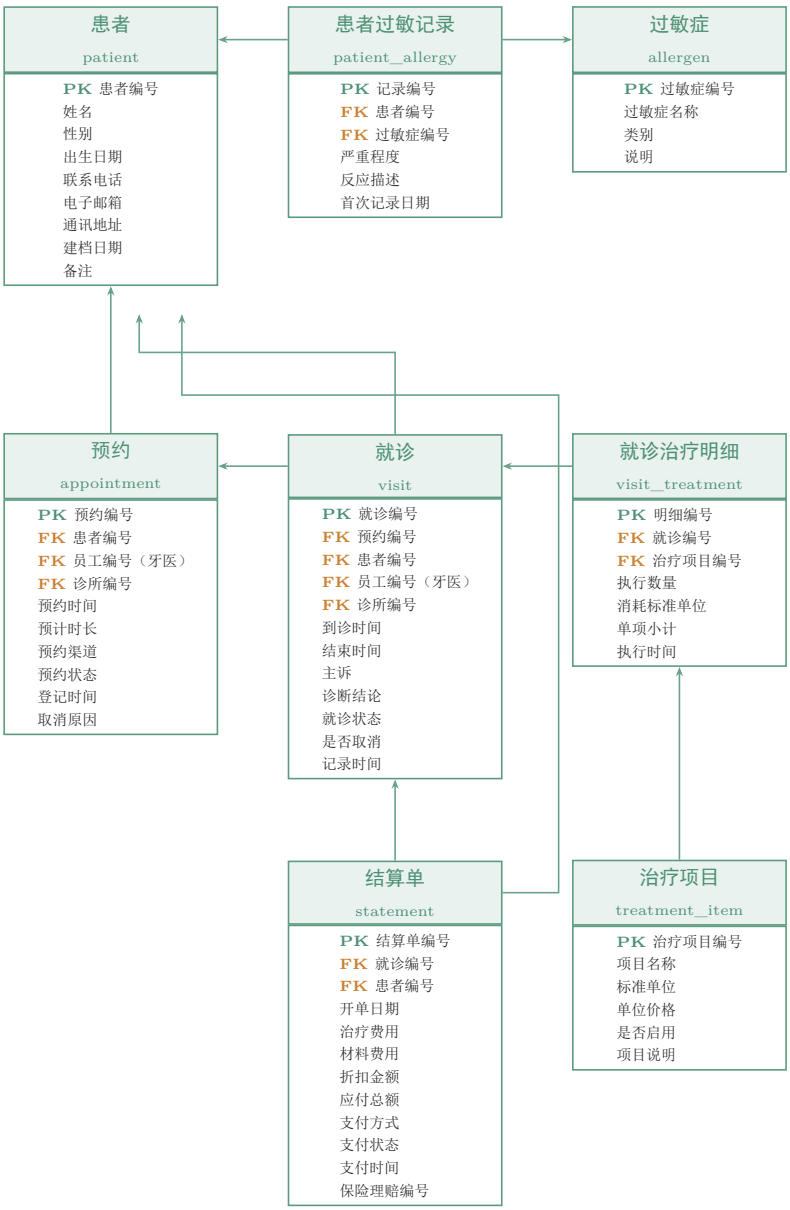


图 11　数据库图表（组织与人员域）：表、主键与外键

图 11 说明　这是图 6 的直接投影。三张人员表的**主键就是员工表的主键**，这正是“泛化 → 超类主键作为所有子类主键”那条规则的落地方式；前台接待员表的诊所编号、牙医执业点表的员工编号与诊所编号、牙医资质表的员工编号与资质编号，都是“外键放在非 1 方”的结果。特别值得一提的是牙医执业点表：它在数据库里是一张典型的**关联表**，两张核心表（牙医、诊所）的多对多关系由它承担，同时它还保留了排班说明与是否主要执业点两个业务列——关联表带业务属性，正是案例中“牙医跨诊所轮岗”这一诉求的存储方式。

6.8 数据库图表（二）：患者与诊疗域



PK 主键　FK 外键　箭头由持外键的表指向被引用的表。
预约与就诊中的员工编号、诊所编号，其被引用表见图 11。

图 12　数据库图表（患者与诊疗域）：表、主键与外键

图 12 说明 这幅图是三张关联表的集中展示。患者过敏记录表是患者与过敏症之间多对多关系的落点，两张外键列 patient_id、allergen_id 之外，还带着严重程度与反应描述；就诊治疗明细表是就诊与治疗项目之间多对多关系的落点，它同时受组合规则的约束，visit_id 必须非空；而预约表、就诊表上的 patient_id、employee_id、clinic_id，则分别来自“患者 → 预约/就诊”“牙医 → 预约/就诊”“诊所 → 预约/就诊”三组一对多关联。整幅图里没有一处冗余列：每一列要么是主键，要么是某个关联的外键，要么是该实体自身携带的领域属性——这正是对象建模正确时映射结果自然满足 3NF 的原因。

6.9 数据字典

两份图表给出了表与表之间的关系，下表进一步给出每张表每一列的数据类型与约束，作为实施阶段的直接依据。

表 16 数据字典（一）：组织与人员域

列名	数据类型	约束	中文含义
employee 员工			
employee_id	INT	PK，自增	员工编号（代理主键）
full_name	VARCHAR(30)	NOT NULL	姓名
gender	CHAR(1)	NOT NULL	性别（M / F）
phone	VARCHAR(20)	NOT NULL	联系电话
hire_date	DATE	NOT NULL	入职日期
status	VARCHAR(10)	NOT NULL	在职状态（在职 / 离职）
dentist 牙医			
employee_id	INT	PK，FK→employee	员工编号（兼作主键与外键）
license_no	VARCHAR(30)	NOT NULL，UNIQUE	执业证书编号
specialty	VARCHAR(30)		专业方向
title	VARCHAR(20)		职称
receptionist 前台接待员			
employee_id	INT	PK，FK→employee	员工编号（兼作主键与外键）
clinic_id	INT	FK→clinic	所属诊所编号
grade	VARCHAR(10)		岗位等级
scope	VARCHAR(50)		可受理业务范围
administrator 总部管理者			
employee_id	INT	PK，FK→employee	员工编号（兼作主键与外键）
position_name	VARCHAR(30)	NOT NULL	职务名称
scope	VARCHAR(50)		分管范围

列名	数据类型	约束	中文含义
clinic　诊所			
clinic_id	INT	PK，自增	诊所编号（代理主键）
clinic_name	VARCHAR(40)	NOT NULL	诊所名称
city	VARCHAR(20)	NOT NULL	所在城市
address	VARCHAR(80)		详细地址
phone	VARCHAR(20)		联系电话
open_date	DATE		开业日期
chair_count	INT		诊椅数量
status	VARCHAR(10)	NOT NULL	营业状态（营业 / 筹备）
dentist_clinic　牙医执业点（关联表）			
dc_id	INT	PK，自增	执业点编号（代理主键）
employee_id	INT	FK→dentist	牙医编号
clinic_id	INT	FK→clinic	诊所编号
schedule_note	VARCHAR(50)		排班说明
is_primary	TINYINT	NOT NULL，默认 0	是否主要执业点
employee_id + clinic_id	—	UNIQUE	同一牙医在同一诊所只允许一条记录
qualification　资质			
qualification_id	INT	PK，自增	资质编号（代理主键）
qualification_name	VARCHAR(40)	NOT NULL	资质名称
issuing_authority	VARCHAR(60)		颁发机构
valid_until	DATE		有效期至
dentist_qualification　牙医资质（关联表）			
dq_id	INT	PK，自增	资质记录编号（代理主键）
employee_id	INT	FK→dentist	牙医编号
qualification_id	INT	FK→qualification	资质编号
obtain_date	DATE		取得日期
certificate_no	VARCHAR(30)		证书编号
employee_id + qualification_id	—	UNIQUE	同一项资质对同一牙医只登记一次

表 17　数据字典（二）：患者与诊疗域

列名	数据类型	约束	中文含义
patient　患者			
patient_id	INT	PK，自增	患者编号（代理主键）
full_name	VARCHAR(30)	NOT NULL	姓名
gender	CHAR(1)	NOT NULL	性别（M / F）
birth_date	DATE		出生日期
phone	VARCHAR(20)	NOT NULL	联系电话
email	VARCHAR(50)		电子邮箱（识别回头客的首选字段）
address	VARCHAR(80)		通讯地址
created_date	DATE	NOT NULL	建档日期
note	VARCHAR(120)		备注
allergen　过敏症			
allergen_id	INT	PK，自增	过敏症编号（代理主键）
allergen_name	VARCHAR(30)	NOT NULL，UNIQUE	过敏症名称
category	VARCHAR(20)		类别（药物 / 食物 / 材料）
description	VARCHAR(120)		说明
patient_allergy　患者过敏记录（关联表）			
record_id	INT	PK，自增	记录编号（代理主键）
patient_id	INT	FK→patient	患者编号
allergen_id	INT	FK→allergen	过敏症编号
severity	VARCHAR(10)	NOT NULL	严重程度（轻 / 中 / 重）
reaction	VARCHAR(120)		反应描述
first_record_date	DATE		首次记录日期
appointment　预约			
appointment_id	INT	PK，自增	预约编号（代理主键）
patient_id	INT	FK→patient	患者编号
employee_id	INT	FK→dentist	接诊牙医编号
clinic_id	INT	FK→clinic	诊所编号
appointment_time	DATETIME	NOT NULL	预约时间
expected_minutes	INT		预计时长（分钟）
channel	VARCHAR(10)		预约渠道（电话 / 到店 / 线上）
status	VARCHAR(10)	NOT NULL	预约状态（已预约 / 已到诊 / 已取消）

列名	数据类型	约束	中文含义
created_time	DATETIME	NOT NULL	登记时间
cancel_reason	VARCHAR(60)		取消原因
visit　就诊			
visit_id	INT	PK，自增	就诊编号（代理主键）
appointment_id	INT	FK→appointment，UNIQUE	来源预约（一次预约最多对应一次就诊）
patient_id	INT	FK→patient	患者编号
employee_id	INT	FK→dentist	接诊牙医编号
clinic_id	INT	FK→clinic	诊所编号
arrival_time	DATETIME	NOT NULL	到诊时间
end_time	DATETIME		结束时间
chief_complaint	VARCHAR(200)		主诉
diagnosis	VARCHAR(200)		诊断结论
status	VARCHAR(10)	NOT NULL	就诊状态（已预约 / 已就诊 / 已取消）
is_cancelled	TINYINT	NOT NULL，默认 0	是否取消（取消率统计标记）
recorded_time	DATETIME	NOT NULL	记录时间
treatment_item　治疗项目			
item_id	INT	PK，自增	治疗项目编号（代理主键）
item_name	VARCHAR(40)	NOT NULL，UNIQUE	项目名称
standard_unit	DECIMAL(6,2)	NOT NULL	标准单位（计价单位，如“颗”“次”）
unit_price	DECIMAL(10,2)	NOT NULL	单位价格
is_active	TINYINT	NOT NULL，默认 1	是否启用
description	VARCHAR(120)		项目说明
visit_treatment　就诊治疗明细（关联表）			
vt_id	INT	PK，自增	明细编号（代理主键）
visit_id	INT	FK→visit，NOT NULL	就诊编号（组合关系，不可为空）
item_id	INT	FK→treatment_item	治疗项目编号
quantity	DECIMAL(6,2)	NOT NULL	执行数量
consumed_units	DECIMAL(8,2)	NOT NULL	消耗标准单位
subtotal	DECIMAL(10,2)	NOT NULL	单项小计
executed_time	DATETIME		执行时间
statement　结算单			

列名	数据类型	约束	中文含义
statement_id	INT	PK，自增	结算单编号（代理主键）
visit_id	INT	FK→visit，UNIQUE	就诊编号（一次就诊最多一张结算单）
patient_id	INT	FK→patient	患者编号
issue_date	DATE	NOT NULL	开单日期
treatment_fee	DECIMAL(10,2)	NOT NULL，默认 0	治疗费用
material_fee	DECIMAL(10,2)	NOT NULL，默认 0	材料费用
discount	DECIMAL(10,2)	默认 0	折扣金额
total_due	DECIMAL(10,2)	NOT NULL	应付总额
payment_method	VARCHAR(20)		支付方式
payment_status	VARCHAR(10)	NOT NULL	支付状态（未付 / 已付 / 部分）
payment_time	DATETIME		支付时间
claim_no	VARCHAR(30)		保险理赔编号

6.10 建表脚本

映射结果可以直接写成数据定义语句。下表按“被引用表先建”的顺序给出 16 张表的建表脚本，字段命名与数据字典一一对应；注释中的中文名称即数据字典里的中文含义，便于开发时对照。

```
CREATE TABLE employee (
  employee_id  INT AUTO_INCREMENT PRIMARY KEY COMMENT '员工编号',
  full_name    VARCHAR(30) NOT NULL COMMENT '姓名',
  gender       CHAR(1)     NOT NULL COMMENT '性别',
  phone        VARCHAR(20) NOT NULL COMMENT '联系电话',
  hire_date    DATE        NOT NULL COMMENT '入职日期',
  status       VARCHAR(10) NOT NULL COMMENT '在职状态'
) COMMENT '员工';

CREATE TABLE clinic (
  clinic_id    INT AUTO_INCREMENT PRIMARY KEY COMMENT '诊所编号',
  clinic_name  VARCHAR(40) NOT NULL COMMENT '诊所名称',
  city         VARCHAR(20) NOT NULL COMMENT '所在城市',
  address      VARCHAR(80) COMMENT '详细地址',
  phone        VARCHAR(20) COMMENT '联系电话',
  open_date    DATE        COMMENT '开业日期',
  chair_count  INT         COMMENT '诊椅数量',
  status       VARCHAR(10) NOT NULL COMMENT '营业状态'
) COMMENT '诊所';

CREATE TABLE qualification (
  qualification_id   INT AUTO_INCREMENT PRIMARY KEY COMMENT '资质编号',
  qualification_name VARCHAR(40) NOT NULL COMMENT '资质名称',
  issuing_authority  VARCHAR(60) COMMENT '颁发机构',
  valid_until        DATE        COMMENT '有效期至'
) COMMENT '资质';
```

```
CREATE TABLE dentist (
  employee_id  INT PRIMARY KEY COMMENT '员工编号',
  license_no   VARCHAR(30) NOT NULL UNIQUE COMMENT '执业证书编号',
  specialty    VARCHAR(30) COMMENT '专业方向',
  title        VARCHAR(20) COMMENT '职称',
  FOREIGN KEY (employee_id) REFERENCES employee(employee_id)
) COMMENT '牙医';

CREATE TABLE receptionist (
  employee_id  INT PRIMARY KEY COMMENT '员工编号',
  clinic_id    INT COMMENT '所属诊所',
  grade        VARCHAR(10) COMMENT '岗位等级',
  scope        VARCHAR(50) COMMENT '可受理业务范围',
  FOREIGN KEY (employee_id) REFERENCES employee(employee_id),
  FOREIGN KEY (clinic_id)   REFERENCES clinic(clinic_id)
) COMMENT '前台接待员';

CREATE TABLE administrator (
  employee_id    INT PRIMARY KEY COMMENT '员工编号',
  position_name  VARCHAR(30) NOT NULL COMMENT '职务名称',
  scope          VARCHAR(50) COMMENT '分管范围',
  FOREIGN KEY (employee_id) REFERENCES employee(employee_id)
) COMMENT '总部管理者';

CREATE TABLE dentist_clinic (
  dc_id         INT AUTO_INCREMENT PRIMARY KEY COMMENT '执业点编号',
  employee_id   INT NOT NULL COMMENT '牙医',
  clinic_id     INT NOT NULL COMMENT '诊所',
  schedule_note VARCHAR(50) COMMENT '排班说明',
  is_primary    TINYINT NOT NULL DEFAULT 0 COMMENT '是否主要执业点',
  UNIQUE KEY uk_dentist_clinic (employee_id, clinic_id),
  FOREIGN KEY (employee_id) REFERENCES dentist(employee_id),
  FOREIGN KEY (clinic_id)   REFERENCES clinic(clinic_id)
) COMMENT '牙医执业点';

CREATE TABLE dentist_qualification (
  dq_id             INT AUTO_INCREMENT PRIMARY KEY COMMENT '资质记录编号',
  employee_id       INT NOT NULL COMMENT '牙医',
  qualification_id  INT NOT NULL COMMENT '资质',
  obtain_date       DATE COMMENT '取得日期',
  certificate_no    VARCHAR(30) COMMENT '证书编号',
  UNIQUE KEY uk_dentist_qual (employee_id, qualification_id),
  FOREIGN KEY (employee_id)      REFERENCES dentist(employee_id),
  FOREIGN KEY (qualification_id) REFERENCES qualification(qualification_id)
) COMMENT '牙医资质';
```

```
CREATE TABLE patient (
  patient_id   INT AUTO_INCREMENT PRIMARY KEY COMMENT '患者编号',
  full_name    VARCHAR(30) NOT NULL COMMENT '姓名',
  gender       CHAR(1)     NOT NULL COMMENT '性别',
  birth_date   DATE        COMMENT '出生日期',
  phone        VARCHAR(20) NOT NULL COMMENT '联系电话',
  email        VARCHAR(50) COMMENT '电子邮箱',
  address      VARCHAR(80) COMMENT '通讯地址',
```

```
  created_date DATE        NOT NULL COMMENT '建档日期',
  note         VARCHAR(120) COMMENT '备注'
) COMMENT '患者';

CREATE TABLE allergen (
  allergen_id    INT AUTO_INCREMENT PRIMARY KEY COMMENT '过敏症编号',
  allergen_name  VARCHAR(30) NOT NULL UNIQUE COMMENT '过敏症名称',
  category       VARCHAR(20) COMMENT '类别',
  description    VARCHAR(120) COMMENT '说明'
) COMMENT '过敏症';

CREATE TABLE treatment_item (
  item_id       INT AUTO_INCREMENT PRIMARY KEY COMMENT '治疗项目编号',
  item_name     VARCHAR(40)  NOT NULL UNIQUE COMMENT '项目名称',
  standard_unit DECIMAL(6,2) NOT NULL COMMENT '标准单位',
  unit_price    DECIMAL(10,2) NOT NULL COMMENT '单位价格',
  is_active     TINYINT NOT NULL DEFAULT 1 COMMENT '是否启用',
  description   VARCHAR(120) COMMENT '项目说明'
) COMMENT '治疗项目';

CREATE TABLE appointment (
  appointment_id   INT AUTO_INCREMENT PRIMARY KEY COMMENT '预约编号',
  patient_id       INT NOT NULL COMMENT '患者',
  employee_id      INT NOT NULL COMMENT '接诊牙医',
  clinic_id        INT NOT NULL COMMENT '诊所',
  appointment_time DATETIME NOT NULL COMMENT '预约时间',
  expected_minutes INT COMMENT '预计时长',
  channel          VARCHAR(10) COMMENT '预约渠道',
  status           VARCHAR(10) NOT NULL COMMENT '预约状态',
  created_time     DATETIME NOT NULL COMMENT '登记时间',
  cancel_reason    VARCHAR(60) COMMENT '取消原因',
  KEY idx_dentist_time (employee_id, appointment_time),
  FOREIGN KEY (patient_id)  REFERENCES patient(patient_id),
  FOREIGN KEY (employee_id) REFERENCES dentist(employee_id),
  FOREIGN KEY (clinic_id)   REFERENCES clinic(clinic_id)
) COMMENT '预约';

CREATE TABLE visit (
  visit_id         INT AUTO_INCREMENT PRIMARY KEY COMMENT '就诊编号',
  appointment_id   INT UNIQUE COMMENT '来源预约',
  patient_id       INT NOT NULL COMMENT '患者',
  employee_id      INT NOT NULL COMMENT '接诊牙医',
  clinic_id        INT NOT NULL COMMENT '诊所',
  arrival_time     DATETIME NOT NULL COMMENT '到诊时间',
  end_time         DATETIME COMMENT '结束时间',
  chief_complaint  VARCHAR(200) COMMENT '主诉',
  diagnosis        VARCHAR(200) COMMENT '诊断结论',
  status           VARCHAR(10) NOT NULL COMMENT '就诊状态',
  is_cancelled     TINYINT NOT NULL DEFAULT 0 COMMENT '是否取消',
  recorded_time    DATETIME NOT NULL COMMENT '记录时间',
  KEY idx_clinic_time (clinic_id, recorded_time),
  FOREIGN KEY (appointment_id) REFERENCES appointment(appointment_id),
  FOREIGN KEY (patient_id)     REFERENCES patient(patient_id),
  FOREIGN KEY (employee_id)    REFERENCES dentist(employee_id),
  FOREIGN KEY (clinic_id)      REFERENCES clinic(clinic_id)
) COMMENT '就诊';
```

```
65
66  CREATE TABLE visit_treatment (
67    vt_id           INT AUTO_INCREMENT PRIMARY KEY COMMENT '明细编号',
68    visit_id        INT NOT NULL COMMENT '就诊',
69    item_id         INT NOT NULL COMMENT '治疗项目',
70    quantity        DECIMAL(6,2)  NOT NULL COMMENT '执行数量',
71    consumed_units  DECIMAL(8,2)  NOT NULL COMMENT '消耗标准单位',
72    subtotal        DECIMAL(10,2) NOT NULL COMMENT '单项小计',
73    executed_time   DATETIME COMMENT '执行时间',
74    FOREIGN KEY (visit_id) REFERENCES visit(visit_id),
75    FOREIGN KEY (item_id)  REFERENCES treatment_item(item_id)
76  ) COMMENT '就诊治疗明细';
77
78  CREATE TABLE patient_allergy (
79    record_id         INT AUTO_INCREMENT PRIMARY KEY COMMENT '记录编号',
80    patient_id        INT NOT NULL COMMENT '患者',
81    allergen_id       INT NOT NULL COMMENT '过敏症',
82    severity          VARCHAR(10) NOT NULL COMMENT '严重程度',
83    reaction          VARCHAR(120) COMMENT '反应描述',
84    first_record_date DATE COMMENT '首次记录日期',
85    UNIQUE KEY uk_patient_allergen (patient_id, allergen_id),
86    FOREIGN KEY (patient_id)  REFERENCES patient(patient_id),
87    FOREIGN KEY (allergen_id) REFERENCES allergen(allergen_id)
88  ) COMMENT '患者过敏记录';
89
90  CREATE TABLE statement (
91    statement_id   INT AUTO_INCREMENT PRIMARY KEY COMMENT '结算单编号',
92    visit_id       INT UNIQUE COMMENT '就诊',
93    patient_id     INT NOT NULL COMMENT '患者',
94    issue_date     DATE NOT NULL COMMENT '开单日期',
95    treatment_fee  DECIMAL(10,2) NOT NULL DEFAULT 0 COMMENT '治疗费用',
96    material_fee   DECIMAL(10,2) NOT NULL DEFAULT 0 COMMENT '材料费用',
97    discount       DECIMAL(10,2) DEFAULT 0 COMMENT '折扣金额',
98    total_due      DECIMAL(10,2) NOT NULL COMMENT '应付总额',
99    payment_method VARCHAR(20) COMMENT '支付方式',
100   payment_status VARCHAR(10) NOT NULL COMMENT '支付状态',
101   payment_time   DATETIME COMMENT '支付时间',
102   claim_no       VARCHAR(30) COMMENT '保险理赔编号',
103   FOREIGN KEY (visit_id)   REFERENCES visit(visit_id),
104   FOREIGN KEY (patient_id) REFERENCES patient(patient_id)
105 ) COMMENT '结算单';
```

6.11 范式检查

按课件的判断，只要对象建模是正确的，映射结果自然满足第一至第三范式；本案逐项核对如下。

表 18　范式核对

范式	要求	本案的核对结果
1NF	每列取值原子，无重复组	过敏史没有挤进患者表，而是独立为 patient_allergy；一名患者有多项过敏症时增加行数而非增加列
2NF	非主属性完全依赖于主键	所有表都使用单列代理主键，不存在“只依赖复合主键的一部分”的列；关联表用唯一约束而非复合主键来保证语义
3NF	消除传递依赖	诊所的城市、地址只存放在 clinic 表，不冗余到 employee 或 visit；治疗项目的单位价格只存放在 treatment_item，就诊治疗明细只存消耗单位与小计，不复制单价

6.12 设计约束与索引安排

表 19　数据层的关键设计决定

设计决定	做法	理由
主键	全部使用代理主键（自增编号）	满足“唯一且恒定”，且把领域字段与主键分开，手机号、邮箱变更不影响引用关系
唯一约束	执业证书编号、项目名称、过敏症名称、预约与结算单的一对一外键均加唯一约束	把业务规则写进数据结构，而不只写在用例规约里
外键完整性	全部显式声明外键	明细不会出现无主记录，医生不会挂到不存在的诊所
金额精度	使用 DECIMAL(10,2)，不使用浮点类型	治疗按“标准单位 × 单位价格”计价，必须精确
时间类型	预约、到诊用 DATETIME；日期用 DATE	取消率、办理时长等度量指标都以时间字段为基础
索引	预约表按（牙医编号 + 预约时间）建复合索引；就诊表按（诊所编号 + 记录时间）建复合索引	支撑排班冲突检查与按诊所按时段的经营报表
敏感信息	不保存银行卡号等支付凭证	结算通过支付网关接口完成，与 UC-04 的设计约束一致

以上两份图表、数据字典与建表脚本构成了设计阶段“建立数据层”的完整交付：分析类图中的每一个类都能在表结构中找到对应，每一条关联都能在外键（或关联表）中找到落点，反之

表结构里也找不出一个没有出处的列。

6.13 精化业务层：从责任到操作

分析类图只回答了“谁负责”，没有回答“怎么算”。精化业务层要做三件事：给实体类补上**领域操作**（而不是一堆 get/set）、把状态机里的每条迁移落到具体操作上、确定层与层之间的依赖方向。课程反复强调的一条底线是：控制类只分配责任，不要把后面的类降格成“数据”——如果业务方法全堆在控制类里，实体类就退化成了一张表。

这条做法在方法论上叫**领域驱动设计**（DDD），两条主张值得单独点出：其一，业务逻辑以**领域对象**为中心，而不是以数据库表为中心——先问“预约这个概念自己能做什么”，再问它存在哪张表；其二，**领域对象不负责自己的持久化**，存取交给映射器或仓储，两者以接口分离。落到本系统，可以识别出三个**聚合**：预约、就诊、结算单。聚合的含义是“一个整体，外部只能通过聚合根修改它”——就诊治疗明细不能脱离就诊单独新增或删除，只能通过 `Visit.addTreatment()` 追加（对应第六章中 visit_id 的 NOT NULL 组合外键）；结算单的状态也只能由 `Statement` 自己的操作迁移，不能被界面直接赋值。识别聚合的实际好处是给事务划边界：一次 `addTreatment` 就是一个事务，不会出现“明细写进去了、就诊没写成”的半截状态。

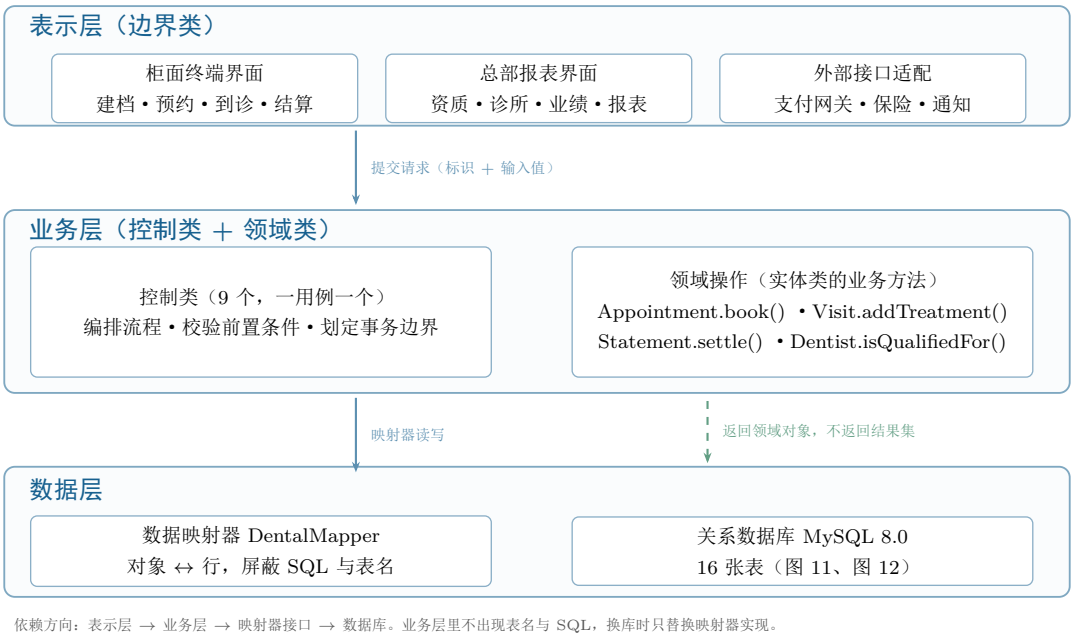


图 13　ADC 系统的分层与依赖方向

图 13 说明　三个层的划分与图 7 的三种构造型一一对应：边界类构成表示层，控制类与实体类的领域操作构成业务层，实体类的持久化由映射器落到数据层。关键在中间那条虚线——映射器返回的是领域对象而不是 ResultSet，业务层因此不知道自己是存在 MySQL 还是别的存储里；这也正是课件里“建立数据层”之后还要“精化业务层”的原因：数据层解决存哪儿，业务层解决谁来保证规则。

表 20　核心领域操作与责任分配

类	关键操作（签名）	责任与业务规则	状态变化
预约	`book(patientId, dentistId, clinicId, time, needs)`	校验同一牙医 30 分钟时段唯一、患者未到诊次数未超阈值，生成预约记录	新增，待就诊
预约	`cancel(reason, operator)`	仅当前时间早于预约时间时允许；记录取消原因与操作人	待就诊 → 已取消
预约	`markAttended()` / `markNoShow()`	由 UC-05 调用；未到诊由日终批处理按“时段已过且无就诊”判定	待就诊 → 已就诊 / 未到诊
牙医	`isQualifiedFor(itemId, date)`	资质类别覆盖治疗项目，且证书在有效期内	只读
牙医	`availableSlots(date, duration)`	排班时段与已占用时段做差集，供预约界面选择	只读
就诊	`start(appointmentId, complaint)`	由预约生成就诊，写入主诉，一个预约至多一个就诊	新增，进行中
就诊	`addTreatment(itemId, units)`	校验项目属于标准目录，取标准单价，生成明细并计算小计	组合明细
就诊	`checkAllergyConflict(itemId)`	比对患者过敏症与治疗材料成分，冲突即拒绝	只读（可抛异常）
就诊	`complete()`	明细非空才允许完成；完成后触发 UC-05 改写预约状态	进行中 → 已完成
结算单	`issue(visitId)`	汇总明细、折扣与保险，生成待出具单据	新增，待出具
结算单	`confirm()` `submitForPayment(method)`	确认后不可直接修改；提交收款时冻结金额	待出具 → 已出具 → 待支付
结算单	`settle(paidAmount, txnId)`	以支付网关返回的交易号为准；金额不符则拒绝	待支付 → 已结算

类	关键操作（签名）	责任与业务规则	状态变化
业绩	`calculate (dentistId, quarter)`	汇总该季度收入、材料成本与分摊运营费用	只读

这张表是“分析到设计”的落点：左列全部来自图 5、图 6 的实体类，右列的状态变化全部来自图 10 的状态机，中间一列则是用例规约里的业务规则。三者能对上，说明前面的模型没有互相打架。特别说明两点：操作名用领域动词（`book`、`settle`、`markNoShow`），不用 `setStatus`；查询类操作与修改类操作分开，前者不改变状态，后者必须守卫条件成立才能执行。

6.14 精化表示层：边界类的界面责任

表示层的边界由分析阶段的边界类划定：它只负责输入采集、输出呈现与简单过滤，**不做业务判断**。这条边界一旦松动，界面就会开始“顺手校验一下”，规则便从业务层漏到表示层，换一个终端（比如将来的患者自助端）就得重写一遍。ADC 的终端只有两类人：前台与总部管理者，因此窗体导航可以画成两棵树。

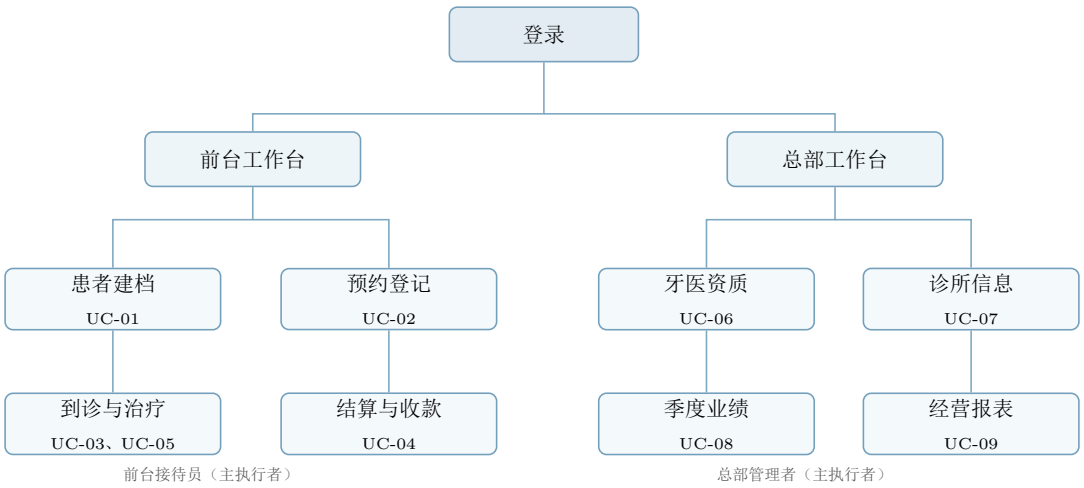


图 14　表示层的窗体导航图

图 14 说明 九个人工操作用例各自对应一个窗体，没有“万能界面”；后台批处理（UC-08 的季度汇总、逾期未到诊判定）不占窗体，由时间触发。导航只有两层——工作台选业务域，业务域选窗体——前台完成一次标准预约最多四次点击进入表单，与 UC-02 质量需求里“不超过 6 次人机交互”的指标相容。

表 21　窗体的输入、输出与表示层约束

窗体	用例	主要输入	主要输出	约束
患者建档	UC-01	姓名、出生日期、电话、地址、过敏症逐项	患者编号、过敏提示	必填项就地校验；过敏症不得只填“有”
预约登记	UC-02	姓名 / 电话、日期时段、治疗需求、牙医	可约时段、牙医列表、预约编号	只读展示单价与状态，不允许改价或改状态
到诊与治疗	UC-03	预约编号、主诉、治疗项目与单位数量	过敏警示、治疗清单与试算金额	单价来自目录，界面不可编辑
结算与收款	UC-04	支付方式、折扣、保险信息	结算单明细、应付总额、支付结果	金额只读；失败原因原样呈现
资质维护	UC-06	证书编号、类别、有效期	资质列表与到期提醒	到期前 60 天高亮
诊所信息	UC-07	名称、地址、诊椅数、营业状态	诊所清单	关闭诊所前校验未来预约
季度业绩	UC-08	季度、牙医	收入、成本、运营费用、激励金额	只读报表，可导出
经营报表	UC-09	时间范围、诊所、治疗类别	取消率、治疗量、收入对比	只读；跨诊所口径统一

把图 14 与上表合起来看，表示层的职责就清楚了：它把“人怎么操作”翻译成“业务层的一个操作调用”，再把返回的领域对象渲染成表格与提示。所有判断——时段是否冲突、单价是否被授权调整、结算单能否红冲——都不在这一层。课件在讲可用性需求时举过“前台 5 次击键完成客人入住”的例子，ADC 的对应指标就写进了 UC-02 的质量需求：熟练前台完成一次标准预约不超过 6 次人机交互。

6.14.1 技术结构：B/S、组件化与框架化

前面讲的是表示层“该做什么”，这里补上“用什么做”。ADC 的终端分布在各诊所柜面与总部办公室，诊所数量还要从 15 家增到 30 家，因此不采用需要逐台安装升级的桌面客户端，而采用 **B/S 架构**：浏览器访问，新诊所开业只需开通账号，版本永远只有一份。

技术结构按前后端分离组织，三层各司其职：

- **结构层**：HTML5 描述窗体语义（表格就是 table、表单就是 form），CSS3 负责版式与状态配色，JavaScript（TypeScript）负责交互与格式校验。界面样式用样式表控制，不靠图片拼版，改配色只改一处变量。

- **组件层**：Vue 3（或 React）按组件组织界面，路由、状态管理、表单校验由框架提供；一个窗体是一个页面组件，由若干子组件拼成。
- **接口层**：后端以 REST/JSON 暴露业务操作，前端只调用接口，不直连数据库、不拼 SQL、不自行计算金额。

组件化不是把页面拆碎，而是把**重复出现的界面片段**提炼成可复用组件。本系统的窗体里反复出现五类片段：

表 22　表示层复用组件清单

组件	使用位置	职责与约束
患者检索框	建档、预约、到诊、结算	按姓名或电话查档，回填患者信息与过敏提示；只读展示，不提供新建入口
牙医时段选择器	预约登记	展示后端算好的可约时段与牙医，组件本身不做资质判断
治疗行编辑表格	到诊与治疗	追加治疗项目与单位数量；单位从目录中选择，不允许自由输入
金额汇总条	治疗、结算	只读显示小计与总额，**没有编辑态**
状态徽章	预约、就诊、结算单	按状态值着色，**没有点击事件**，不提供改状态的入口

组件化的收益有两层：一是复用，患者检索框只实现一次，四个窗体的行为完全一致；二是**约束**——金额汇总条根本没有编辑态，状态徽章根本没有点击事件，界面上就不存在“改价”“改状态”的入口，业务规则不必依赖前端自觉。

框架化的边界要划清：路由、状态管理、必填与格式校验、权限显示交给框架；**业务判断一律不交给前端**。前端可以做“日期不能早于今天”这类格式与范围校验，不能做“该时段是否已被占用”“这张结算单能不能红冲”——后者必须由业务层判定，因为前端校验总能被绕过。权限同理：前端路由只决定“看不看得到”，后端接口必须再校验一次“能不能做”，前端从来不是防线。

最后是设备与可用性：诊椅旁可能用平板，触屏下按钮要大、输入要少，因此治疗行的录入以“选项目 + 选常用单位”为主，而不是自由文本；这与 UC-03 的质量需求“录入一项治疗不超过 4 次交互”是同一件事的两种表述。